VBZS FileMaker Plug-in

Function Reference Manual

Menno van Beek

8 November 2026

Contents

Function Reference
Function Groups
General Conventions
XML Support
Versioning
License
VBZS_AddWorkingdays
- Description
- Syntax
- Parameters
- Result
- Examples
- Invalid working-week patterns
- Notes
- Design philosophy

VBZS_Ascii
- Description
- Syntax
- Parameters
- Default filter
- Email filter
- XML filter
- Carriage returns and whitespace
- BOM removal
- Unsupported characters
- Examples
- Unknown filter types
- Note!
- Design philosophy

VBZS_DateFormat
- Description
- Syntax
- Parameters
- Format codes
- Literal characters
- Examples
- Localized day and month names
- Lowercase and proper case names

- Creating custom formats
- Notes
- Design philosophy

VBZS_Easter

- Description
- Syntax
- Parameters
- Result
- Example
- Supported range
- Notes
- Design philosophy

VBZS_Fraction

- Description
- Syntax
- Parameters
- Result
- Examples
- Fixed-length results
- Truncation and rounding
- Invalid `digits` values
- Notes
- Design philosophy

VBZS_FrontTabPanels

- Description
- Syntax
- Requirements
- Parameters
- Result
- Without a parameter
- Nested panels
- Filtering visible panels
- Parameter order
- Similarity to FilterValues
- Example: testing selected panels
- Using the function as a visibility test
- Notes
- Design philosophy

VBZS_JSONParameters

- Description
- Syntax

- Parameters
- Result
- Processing a JSON object
- Using namespaces
- Variable scope
- Nested objects and arrays
- Root-level JSON arrays
- Array indexing and FileMaker repetitions
- `max_repetition`
- Processing variable-length arrays
- Avoiding artifacts from earlier arrays
- Passing structured data between scripts
- Converting JSON keys to FileMaker variable names
- Parentheses and repetitions
- Maximum root-level elements
- Error handling
- Notes
- Design philosophy

VBZS_ListLogic

- Description
- Syntax
- Parameters
- Result
- Case-insensitive comparison
- AND
- XOR
- NOT
- List order
- Logical overview
- Notes
- Design philosophy

VBZS_Local2UTC

- Description
- Syntax
- Parameters
- Daylight Saving Time
- Standard format
- Format `1`
- ANSI format
- Database format
- Full ISO 8601 format
- UTC/Zulu format

- `F` versus `Z`
- Fractional timezone offsets
- Regional limitation
- Notes
- Design philosophy

VBZS_ModKeys

- Description
- Syntax
- Parameters
- Result
- Created variables
- Example without a namespace
- Example with a namespace
- Global variables
- Platform behaviour
- Evaluation time
- Notes
- Design philosophy

VBZS_OntDiakriet

- Description
- Syntax
- Parameters
- Result
- Example
- Diacritic replacement
- Multi-character replacements
- Unsupported characters
- FileMaker searches
- Use with external systems
- Notes
- Design philosophy

VBZS_QualifiedEmailAddresses

- Syntax
- Description
- Parameters
- Result
- Validation
- Examples
- Character sets
- Input normalization
- Notes

- See also

VBZS_ResponseCode

- Description
- Syntax
- Parameters
- Result
- Multiple HTTP responses
- Invalid response header
- Unknown response code
- Supported response messages
- Example
- Response messages
- Notes
- Design philosophy

VBZS_TrimAll_CR

- Description
- Syntax
- Parameters
- Result
- Spaces
- Example
- Notes
- Design philosophy

VBZS_UTC2Local

- Description
- Syntax
- Parameters
- Result
- Example
- Fractional seconds
- Fractional timezone offsets
- Daylight Saving Time
- Relationship with VBZS_Local2UTC
- Notes
- Design philosophy

VBZS_Version

- Description
- Syntax
- Parameters
- Standard version representation
- Auto-update representation

- Version comparison
- Example
- Notes
- Design philosophy

VBZS_WeekFiscalToDate

- Description
- Syntax
- Parameters
- Result
- Example
- Week numbering
- Monday-based ISO week
- Configurable first weekday
- Week-numbering year versus calendar year
- Notes
- Design philosophy

VBZS_XChildren

- Description
- Syntax
- Parameters
- Result
- Example
- Exploring an unknown XML document
- Namespace handling
- Important
- Related functions

VBZS_XPath

- Description
- Syntax
- Parameters
- Result
- Basic example
- Namespace handling
- Important: the input must be well-formed XML
- Important: the result contains raw content
- Working with complex elements
- Recommended workflow
- Design philosophy
- Related functions

VBZS_YearFiscal

- Description

Syntax
Parameters
Result
Example
Week-numbering rule
Monday-based ISO year
Dates at the end of December
Configurable first weekday
Relationship with VBZS_WeekFiscalToDate
Notes
Design philosophy

VBZS is a cross-platform plug-in for FileMaker that provides additional calculation functions for common development tasks.

The plug-in focuses on small, practical functions that simplify operations which are missing from FileMaker, require unnecessarily complex calculations, or are useful enough to provide as native plug-in functions.

This documentation describes the individual VBZS functions, their parameters, behaviour, limitations and practical use.

Function Reference

Functions are listed alphabetically.

Function	Description
VBZS_AddWorkingdays	Adds working days to a date using a configurable working-week pattern.
VBZS_Ascii	Filters text using predefined character sets for general text, email addresses and XML.
VBZS_DateFormat	Converts a FileMaker date to text using a configurable date-format template.
VBZS_Easter	Calculates Easter Sunday for a given year.
VBZS_Fraction	Returns the fractional digits of a number as fixed-length text.
VBZS_FrontTabPanels	Returns the named tab and slide panels currently visible on a FileMaker layout.
VBZS_JSONParameters	Creates FileMaker variables from JSON objects and arrays, with support for namespaces and repeating variables.
VBZS_ListLogic	Compares two lists of values using `AND`, `XOR` or `NOT` logic.
VBZS_Local2UTC	Formats or converts a local timestamp to ISO 8601 or UTC/Zulu representations.
VBZS_ModKeys	Makes the current FileMaker modifier-key states available as separate variables.
VBZS_OntDiakriet	Filters text while replacing supported diacritics and special characters with ASCII equivalents.
VBZS_ResponseCode	Extracts the HTTP response code from an `Insert from URL` response header and creates response variables.
VBZS_QualifiedEmailAddresses	

Function	Description
	Validates one or more email addresses and returns only the addresses that meet the validation rules
VBZS_TrimAll_CR	Removes empty values from a return-delimited FileMaker list without modifying spaces.
VBZS_UTC2Local	Converts a UTC timestamp or numeric UTC value to local time with European DST correction.
VBZS_Version	Returns the installed VBZS version in readable or fixed-length version-comparison format.
VBZS_WeekFiscalToDate	Returns the starting date of a specified week using configurable ISO-style week numbering.
VBZS_XChildren	Returns the direct child elements and attributes of an XML element.
VBZS_XPath	Retrieves content from XML using XPath with simplified namespace handling.
VBZS_YearFiscal	Returns the ISO-style week-numbering year for a given date.

Function Groups

Dates and Times

- VBZS_AddWorkingdays
- VBZS_DateFormat
- VBZS_Easter
- VBZS_Local2UTC
- VBZS_UTC2Local
- VBZS_WeekFiscalToDate
- VBZS_YearFiscal

Text and Lists

- VBZS_Ascii
- VBZS_Fraction
- VBZS_ListLogic
- VBZS_OntDiakriet
- VBZS_QualifiedEmailAddresses
- VBZS_TrimAll_CR

XML

- VBZS_XChildren
- VBZS_XPath

JSON

- VBZS_JSONParameters

FileMaker Interface and Environment

- VBZS_FrontTabPanels
- VBZS_ModKeys

HTTP and Integration

- VBZS_ResponseCode

Plug-in Information

- VBZS_Version

General Conventions

VBZS functions are designed to follow normal FileMaker conventions wherever practical.

Function documentation describes the supported behaviour of each function. An implementation may sometimes accept input outside the documented range or format, but undocumented behaviour should not be relied upon.

Examples use FileMaker calculation syntax and return-delimited lists are represented using the FileMaker paragraph character:

> ¶

XML Support

VBZS provides VBZS_XChildren and VBZS_XPath for working with XML.

The XML functions are designed to make namespace handling straightforward dual documentation files are intentionally independent so they can be read directly from the repository or linked from other documentation.

Versioning

The installed VBZS version can be retrieved using:

```
VBZS_Version ( "" )
```

For automated version comparison:

```
VBZS_Version ( 1 )
```

or:

```
VBZS_Version ( "autoupdate" )
```

See docs/VBZS_Version.md for details.

License

VBZS is proprietary, closed-source software provided free of charge.

This version may be used free of charge in personal, professional and commercial FileMaker solutions under the terms of the VBZS Freeware License.

The source code is not distributed and remains the intellectual property of the author.

The license applies to the VBZS version with which it is distributed. Versions released under the VBZS Freeware License remain licensed under those terms.

Future versions of VBZS may be released under different license terms.

See LICENSE.md for the complete license terms.

VBZS_AddWorkingdays

Description

`VBZS_AddWorkingdays` adds a specified number of working days to a starting date.

By default, Monday through Friday are considered working days. However, not everybody has the same working week. The optional `workdays` parameter allows any day of the week to be defined as either a working day or a non-working day.

This makes the function suitable for both conventional Monday-to-Friday schedules and alternative working patterns where, for example, Saturday or Sunday is a working day and another weekday is not.

The starting date itself does not have to be a working day.

Syntax

```
VBZS_AddWorkingdays ( startdate ; workdays ; daystoadd )
```

Parameters

`startdate`

The FileMaker date from which the calculation starts.

The starting date is not counted as one of the working days to add. Counting starts with the following calendar day.

It is not relevant whether `startdate` itself is a working day.

`workdays`

Defines which days of the week are considered working days.

If `workdays` is empty, the standard Monday-to-Friday working week is used.

When specified, `workdays` must:

- contain exactly 7 characters;
- contain only `0` and `1`;
- contain at least one `1`.

The positions follow FileMaker's weekday order:

Position	Day
1	Sunday
2	Monday
3	Tuesday
4	Wednesday
5	Thursday
6	Friday
7	Saturday

A `1` indicates a working day and a `0` indicates a non-working day.

The standard Monday-to-Friday working week can therefore be represented as:

```
0111110
```

Because this is the default, there is normally no need to specify this pattern explicitly. Leave `workdays` empty instead.

An invalid `workdays` pattern causes the function to return `startdate` unchanged, regardless of which day of the week `startdate` represents.

`daystoadd`

The number of working days to add to `startdate`.

`daystoadd` must be a positive integer for working days to be added.

If `daystoadd` is `0` or negative, the function returns `startdate` unchanged.

Result

Returns the calculated FileMaker date after adding the requested number of working days.

Only days defined as working days by `workdays` are counted.

If `workdays` is empty, Monday through Friday are counted as working days.

Examples

Standard working week

The standard Monday-to-Friday schedule does not require a `workdays` pattern:

```
VBZS_AddWorkingdays (
    Date ( 11 ; 7 ; 2026 ) ;
    "" ;
    1
)
```

November 7, 2026 is a Saturday. The next working day is Monday, November 9.

Result:

```
09/11/2026
```

The same calculation starting on a working day:

```
VBZS_AddWorkingdays (
    Date ( 11 ; 9 ; 2026 ) ;
    "" ;
    1
)
```

returns:

```
10/11/2026
```

The starting date is therefore only the starting point of the calculation. It is never counted as one of the working days being added.

Alternative working week

A different working-week pattern can be specified when required.

For example:

```
0011111
```

defines the following week:

Day	Working day
Sunday	No
Monday	No
Tuesday	Yes
Wednesday	Yes
Thursday	Yes
Friday	Yes
Saturday	Yes

Using this pattern:

```
VBZS_AddWorkingdays (
    startdate ;
    "0011111" ;
    5
)
```

adds five working days according to a Tuesday-through-Saturday schedule.

Invalid working-week patterns

The following patterns are invalid:

```
011110
```

Only six positions are supplied.

```
01111100
```

Eight positions are supplied.

```
011X110
```

A character other than `0` or `1` is present.

```
0000000
```

No working day is defined.

When an invalid pattern is supplied, `VBZS_AddWorkingdays` returns `startdate` unchanged.

Notes

- The default working week is Monday through Friday.
- Leave `workdays` empty to use the default working week.
- The order of the `workdays` pattern is Sunday through Saturday, following FileMaker's weekday convention.
- `startdate` does not have to be a working day.
- `startdate` itself is never counted as one of the days to add.
- A `daystoadd` value of `0` or less returns `startdate` unchanged.
- An invalid `workdays` pattern returns `startdate` unchanged.

Design philosophy

A working day is not necessarily the same for every person, company or application.

Simply excluding Saturday and Sunday assumes a conventional Monday-to-Friday schedule. `VBZS_AddWorkingdays` deliberately avoids making that assumption.

Instead, the function separates the starting date from the definition of the working week. This allows the same calculation to be used for conventional office schedules as well as alternative working patterns, without requiring different calculation logic.

VBZS_Ascii

Description

`VBZS_Ascii` is a simple whitelist-based character filter.

It is intended to clean text that may contain unwanted characters, particularly invisible characters introduced by copy/paste operations, imported data, external systems or character encoding.

The function is deliberately simple. It does not try to interpret, repair or intelligently convert the supplied text. Apart from a few explicitly handled characters, anything that is not allowed by the selected filter is simply removed.

Three filtering modes are available:

- default
- `eml`
- `xml`

Syntax

```
VBZS_Ascii ( text ; type )
```

Parameters

`text`

The text to be filtered.

Unsupported characters are removed from the result.

A few characters receive special treatment:

- non-breaking spaces are converted to normal spaces;
- a BOM (Byte Order Mark) is removed;
- other characters that are not allowed by the selected filter are removed.

`type`

Determines which character filter is used.

The following values are supported:

`type`	Filter
`""`	Default character filter
`"eml"`	Email character filter
`"xml"`	XML character filter

Any other value is ignored and causes the default filter to be used.

Default filter

When `type` is empty, the general-purpose character filter is used:

```
VBZS_Ascii ( text ; "" )
```

The default filter retains:

- upper- and lowercase Latin letters;
- commonly used accented characters;
- digits;
- common punctuation;
- a limited set of symbols;

- spaces;
- tabs;
- FileMaker carriage returns.

The character set includes:

```
abcdefghijklmnopqrstuvwxyz
áàâäãåçéèêëíìîïñóòôöõøúùûüý
ABCDEFGHIJKLMNOPQRSTUVWXYZ
ÁÀÂÄÃÅÇÉÈÊËÍÌÎÏÑÓÒÔÖÕØÚÙÛÜÝ
0123456789
!"#$%&'()*+,-./ :;<=>?@[\]^_`{|}~
¢£¤¥¦§©¬®°±¶
```

Tab characters (`Char ( 9 )`) and FileMaker carriage returns (`Char ( 13 )`) are also preserved.

Line feed characters (`Char ( 10 )`) are removed.

Email filter

Use `eml` to apply a more restrictive filter intended for cleaning text that will be used as an email address:

```
VBZS_Ascii ( text ; "eml" )
```

The email filter retains:

```
abcdefghijklmnopqrstuvwxyz
áàâäãåçéèêëíìîïñóòôöõøúùûüý
ABCDEFGHIJKLMNOPQRSTUVWXYZ
ÁÀÂÄÃÅÇÉÈÊËÍÌÎÏÑÓÒÔÖÕØÚÙÛÜÝ
0123456789
@._
```

Tabs, carriage returns, line feeds, spaces and other punctuation are removed.

> **Important:** `eml` is a character filter, not an email address validator.

The function only removes characters that are not included in its email character set. It does not check the structure of the resulting address.

For example, the presence or position of `.`, `_` or `@` is not validated. A string that passes through the filter is therefore not necessarily a valid email address.

Use `eml` to **clean** an email address, not to validate one.

XML filter

Use `xml` for text containing XML:

```
VBZS_Ascii ( text ; "xml" )
```

The XML filter preserves the broader character set required when filtering XML content, including XML markup characters such as `<` and `>`.

The character set includes:

```
abcdefghijklmnopqrstuvwxyz
áàâäãåçéèêëíìîïñóòôöõøúùûüý
ABCDEFGHIJKLMNOPQRSTUVWXYZ
ÁÀÂÄÃÅÇÉÈÊËÍÌÎÏÑÓÒÔÖÕØÚÙÛÜÝ
0123456789
@._
<>"!#$%&'()*+,-./ :;<=>?@[\]^_`{|}~
¢£¤¥¦§©¬®°±¶
```

Tab characters (`Char( 9 )`) and FileMaker carriage returns (`Char ( 13 )`) are preserved.

Line feed characters (`Char ( 10 )`) are removed.

> **Important:** `xml` is a character filter. It does not validate, parse or repair XML.

Passing text through:

```
VBZS_Ascii ( text ; "xml" )
```

does not guarantee that the resulting text is well-formed XML.

Use the XML mode to remove unwanted characters from XML text, not to correct an invalid XML document.

Carriage returns and whitespace

Whitespace handling depends on the selected mode:

Character	default	`eml`	`xml`
Normal space	Preserved	Removed	Preserved
Non-breaking space	Converted to normal space	Removed	Converted to normal space
Tab (`Char ( 9 )`)	Preserved	Removed	Preserved
Carriage return (`Char ( 13 )`)	Preserved	Removed	Preserved
Line feed (`Char ( 10 )`)	Removed	Removed	Removed

This is particularly relevant when text originates outside FileMaker and contains different line-ending conventions.

BOM removal

A BOM (Byte Order Mark) is removed from the supplied text.

BOM characters can occur when text originates from files of external systems using Unicode encodings. They are normally invisible, but can interfere with comparisons, parsing or other text processing.

`VBZS_Ascii` removes them instead of returning them as part of the cleaned text.

Unsupported characters

Characters not included in the selected filter are simply remomed.

The function does not normally attempt to replace an unsupported character with a visually similar supported character.

For example, unsupported Unicode characters, unusual invisible characters or other characters outside the selected whitelist simply disappear from the result.

This behaviour is intentional.

Examples

General text cleaning

```
VBZS_Ascii ( text ; "" )
```

uses the default character set and removes characters outside that set.

This can be useful when imported or copied text contains unwanted invisible characters that are difficult to locate from within FileMaker.

Cleaning an email address

```
VBZS_Ascii (
    email ;
    "eml"
)
```

removes characters outside the email filter.

The result is cleaned text intended for use as an email address. It is not a guarantee that the result actually represents a valid email address.

Cleaning XML text

```
VBZS_Ascii (
    xml ;
    "xml"
)
```

retains the supported characters used in XML content and markup while removing characters outside the XML filter.

The result is not automatically guaranteed to be well-formed XML.

Unknown filter types

If `type` contains any value other than:

```
""
"eml"
"xml"
```

the value is ignored and the default filter is used.

For example:

```
VBZS_Ascii (
    text ;
    "something"
)
```

behaves as if the calculation were:

```
VBZS_Ascii(
    text ;
    ""
)
```

Note!

- The function uses predefined character whitelists.
- Unsupported characters are removed rather than interpreted.
- Non-breaking spaces are converted to normal spaces where spaces are permitted.
- BOM characters are removed.
- `Char ( 9 )` is preserved by the default and `xml` filters.
- `Char ( 13 )` is preserved by the default and `xml` filters.
- `Char ( 10 )` is removed.
- `xml` cleans characters but does not validate an email address.
- `xml` cleans characters but does not validate or repair XML.
- An unknown `type` value falls back to the default filter.

Design philosophy

`VBZS_Ascii` is deliberately a simple function.

Text obtained from external sources can contain invisible or unexpected characters that are difficult to identify in FileMaker and can cause problems during comparison, parsing, exporting or further processing.

The purpose of `VBZS_Ascii` is not to understand those characters. The function simply rettins a known set of useful characters and removes everything else.

The `eml` and `xml` modes provide more specialised whitelists, but follow exactly the same principle:

keep what is expected, remove what is not, and do not pretend to validate the resulting data.

VBZS_DateFormat

Description

`VBZS_DateFormat` converts a FileMaker date to text using a user-defined formatting template.

The template determines how the day, month and year are represented and which separators or additional characters are included in the result.

Day and month names can also be used. These names are automatically returned in the language of the system locale on the computer where FileMaker is running.

Syntax

```
VBZS_DateFormat ( thedate ; dateformat )
```

Parameters

`thedate`

The FileMaker date to format.

`dateformat`

A template describing how the date should be returned.

The template can contain formatting codes for the individual parts of the date. Characters that are not reserved formatting characters are copied literally to the result.

Format codes

The following formatting codes are available:

Code	Result
`d`	Day number
`dd`	Day number, padded to two digits
`m`	Month number
`mm`	Month number, padded to two digits
`yy`	Year as two digits
`yyyy`	Year as four digits
`T`	Localized day name in lowercase
`TT`	Localized day name in proper case
`B`	Localized month name in lowercase
`BB`	Localized month name in proper case

The reserved formatting characters are not case-sensitive.

For example:

```
dd
DD
```

have the same meaning.

Likewise, the other formatting codes can be entered using either uppercase or lowercase characters.

Literal characters

Characters that are not reserved by the formatting template are copied literally to the result.

This makes it possible to freely choose date separators and formatting.

For example:

```
dd-mm-yyyy
```

produces a date such as:

```
07-11-2026
```

while:

```
dd/mm/yyyy
```

produces:

```
07/11/2026
```

Spaces and other non-reserved characters can also be included directly in the template.

Examples

Numeric date

```
VBZS_DateFormat (
    Date ( 11 ; 7 ; 2026 ) ;
    "dd-mm-yyyy"
)
```

returns:

```
07-11-2026
```

Changing only the literal separators:

```
VBZS_DateFormat (
    Date ( 11 ; 7 ; 2026 ) ;
    "dd/mm/yyyy"
)
```

returns:

```
07/11/2026
```

Different order

The components of the date can be placed in any desired order.

```
VBZS_DateFormat (
    Date ( 11 ; 7 ; 2026 ) ;
    "yyyy-mm-dd"
)
```

returns:

```
2026-11-07
```

Day and month names

Day and month names can be included using `T` and `B`.

For example:

```
VBZS_DateFormat (
    Date ( 11 ; 7 ; 2026 ) ;
    "TT d BB yyyy"
)
```

On a system using an English locale this would return:

```
Saturday 7 November 2026
```

On a system using a Dutch locale, the day and month names are returned in Dutch instead.

Localized day and month names

The following format codes are locale-dependent:

```
T
TT
B
BB
```

`T` and `TT` return the name of the day.

`B` and `BB` return the name of the month.

The language is determined by the system locale of the computer on which FileMaker is running.

VBZS does not contain its own language tables and no language needs to be specified in the function.

> **Important:** The same FileMaker solution may therefore return day and month names in different languages when used on computers with different system locale settings.

This also applies when a calculation is evaluated on FileMaker Server rather than on a FileMaker Pro client. The locale of the system performing the calculation determines the language of the resulting day and month names.

Numeric date components such as `dd`, `mm` and `yyyy` are not affected by this language selection.

Lowercase and proper case names

A single `T` or `B` returns the localized name in lowercase.

For example:

```
T
```

may return:

```
saturday
```

and:

```
B
```

may return:

```
november
```

Using the code twice returns the name in proper format:

```
TT
```

may return:

```
Saturday
```

and:

```
BB
```

may return:

```
November
```

The actual words depend on the system locale.

Creating custom formats

Because non-reserved characters are copied literally, `dateformat` can be used to create many different date representations.

Examples include:

```
dd-mm-yyyy
mm/dd/yyyy
yyyy-mm-dd
d/m/yy
TT d BB yyyy
```

The template controls the representation of the date without changing the underlying FileMaker date value.

Notes

- `VBZS_DateFormat` returns the formatted date as text.
- `thedate` is a FileMaker date.

- `dateformat` defines the resulting text representation.
- `d` and `m` return day and month numbers without forced leading zeros.
- `dd` and `mm` return day and month numbers using two digits.
- `yy` returns a two-digit year.
- `yyyy` returns a four-digit year.
- `T` and `B` return lowercase localized day and month names.
- `TT` and `BB` return localized day and month names in proper format.
- Formatting codes are not case-sensitive.
- Non-reserved characters are copied literally.
- Day and month names use the locale of the system on which FileMaker performs the calculation.

Design philosophy

FileMaker dates are values, but dates often need to be presented as text in a specific format for display, export, communication with another system or use in generated content.

`VBZS_DateFormat` separates the date value from its textual representation by using a small formatting template.

Common numeric date formats can be generated without depending on the date display format of a FileMaker field. At the same time, localized day and month names can be obtained without maintaining separate language tables in the FileMaker solution.

For localized names, the function deliberately uses the locale already available on the system running FileMaker rather than introducing its own translation or language mechanism.

VBZS_Easter

Description

`VBZS_Easter` calculates the date of Easter Sunday for a given year.

The calculation uses Gauss’s Easter algorithm.

Syntax

```
VBZS_Easter ( theYear )
```

Parameters

`theYear`

The year for which Easter Sunday should be calculated.

Result

Returns a FileMaker date containing the calculated date of Easter Sunday for the specified year.

Example

```
VBZS_Easter ( 2026 )
```

returns:

```
05/04/2026
```

Supported range

`VBZS_Easter` provides a reliable Easter date for years from:

```
1583 - 2100
```

The function does not reject years outside this range and may still return a date. A result outside the supported range should not be considered reliable.

> **Important:** A returned date does not imply that the supplied year is within the supported range. Use `VBZS_Easter` for years from 1583 through 2100.

Notes

- The function calculates Easter Sunday.
- The calculation is based on Gauss's Easter algorithm.
- The result is returned as a FileMaker date.
- The supported range is 1583 through 2100.
- Years outside the supported range may produce a result, but that result is not guaranteed to be correct.

Design philosophy

Calculating Easter Sunday requires more than a fixed month and day because its date changes every year.

`VBZS_Easter` provides this calculation as a single FileMaker function, allowing Easter Sunday to be used as the basis for other date calculations.

For example, once Easter Sunday is known, Easter-related dates can be calculated using normal FileMaker date arithmetic.

VBZS_Fraction

Description

`VBZS_Fraction` returns the digits of the fractional part of a number as text.

The result always contains the number of characters specified by `digits`. Leading zeros are preserved and missing digits are added as trailing zeros.

If the fractional part contains more digits than requested, the result is truncated. The function does not round the number.

Syntax

```
VBZS_Fraction ( number ; digits )
```

Parameters

`number`

The number from which the fractional digits are extracted.

The integer part of the number is ignored.

For negative numbers, the minus sign is also ignored. The function returns only the digits of the fractional part.

`digits`

A non-negative integer specifying the exact number of fractional digits to return.

If `digits` is `0`, the function returns empty text.

`digits` must be an integer. Fractional values for `digits` are not supported and may produce unusable results.

Result

Returns the requested number of fractional digits as text.

The result is returned as text rather than as a number so that leading zeros are preserved.

The result is adjusted to exactly the length specified by `digits`:

- if the fractional part contains fewer digits, zeros are added on the right;
- if the fractional part contains more digits, the excess digits are removed;
- no rounding is performed.

Examples

Preserve leading zeros

```
VBZS_Fraction (
    10,0003 ;
    4
)
```

returns:

```
0003
```

The result is text. If it were returned as a number, the leading zeros would be lost.

Add trailing zeros

```
VBZS_Fraction (
    10,12 ;
    4
)
```

returns:

```
1200
```

The fractional part contains only two digits, so two zeros are added on the right.

Negative numbers

```
VBZS_Fraction (
    -10,12 ;
    4
)
```

returns:

```
1200
```

The minus sign and integer part do not affect the result.

Truncate excess digits

```
VBZS_Fraction (
    10,123456 ;
    4
)
```

returns:

```
1234
```

The additional digits are removed. The value is not rounded to `1235`.

Return no fractional digits

```
VBZS_Fraction (
    10,1234 ;
    0
)
```

returns empty text.

Fixed-length results

For the same `digits` value, every result has the same length.

For example, with `digits` set to `4`:

Number	Result
`10,2`	`2000`
`10,12`	`1200`
`10,1234`	`1234`
`10,123456`	`1234`
`10,0003`	`0003`
`-10,12`	`1200`

This makes the function particularly useful when numbers are incorporated into text and every fractional part must have a consistent number of characters.

Truncation and rounding

`VBZS_Fraction` truncates the fractional part when it contains more digits than requested.

It does not round the number.

> **Important:** If rounding is required, round the number before passing it to `VBZS_Fraction`.

For example, this calculation:

```
VBZS_Fraction (
    10,123456 ;
    4
)
```

returns:

```
1234
```

It does not return:

```
1235
```

Invalid `digits` values

`digits` is intended to be a non-negative integer.

A fractional value for `digits` is not a meaningful instruction and may produce an unusable result. Validate or convert `digits` before calling the function when its value comes from user input or an external source.

Notes

- The function returns text.
- Only the fractional digits are returned.
- The integer part is not included.
- The sign of a negative number is not included.
- Leading zeros are preserved.
- Missing digits are added as zeros on the right.
- Excess digits are truncated.
- The function does not round.
- `digits = 0` returns empty text.
- `digits` should be a non-negative integer.

Design philosophy

`VBZS_Fraction` is intended for situations where numbers are incorporated into text and each fractional part must have a consistent length.

Returning the result as text preserves leading zeros, while padding and truncation ensure that every result contains exactly the requested number of characters.

The function performs no rounding because its purpose is to format the existing fractional digits. When rounding is required, the number should be rounded before it is passed to `VBZS_Fraction`.

VBZS_FrontTabPanels

Description

`VBZS_FrontTabPanels` returns the names of the currently visible tab panels and slide panels on the current FileMaker layout.

The function can be used with or without a parameter.

Without a parameter, it returns all named tab and slide panels that are currently visible.

When a parameter is supplied, it acts as a filter and only the visible panels whose names occur in the supplied list are returned.

This makes `VBZS_FrontTabPanels` useful both for discovering the current visible panel structure and for testing whether specific panels are currently visible.

Syntax

```
VBZS_FrontTabPanels ( param )
```

Requirements

Tab panels and slide panels must have an object name assigned in FileMaker's Inspector.

The function identifies panels by their object name. A panel without an object name cannot be returned or matched against `param`.

For this reason, panels that need to be detected by `VBZS_FrontTabPanels` should always be given a unique and meaningful object name.

Parameters

`param`

An optional return-delimited list of panel object names.

If `param` is empty, all currently visible named tab panels and slide panels are returned.

If `param` contains one or more names, only visible panels whose names occur in that list are returned.

The order of the values in `param` does not determine the order of the result.

Result

Returns a return-delimited list containing the object names of the matching visible tab and slide panels.

Nested visible panels are returned from the outermost panel to the innermost panel.

Without a parameter

To retrieve all currently visible named tab and slide panels:

```
VBZS_FrontTabPanels ( "" )
```

Suppose the currently visible panel structure is:

```
Customer
└── Details
    └── Address
```

The result is:

```
Customer
Details
Address
```

All three panels are currently visible and have an object name, so all three are returned.

Nested panels

`VBZS_FrontTabPanels` also detects visible panels that are located inside other visible tab or slide panels.

For example:

```
Customer
└── Details
    └── Address
```

If `Customer`, `Details` and `Address` represent the currently visible route through nested panel objects, all three can be returned.

The order is from outside to inside:

```
Customer
Details
Address
```

This makes it possible to determine more than just the front panel of a single tab control. The function can describe the complete visible panel path through nested panel objects on the layout.

Filtering visible panels

When `param` is supplied, it is treated as a return-delimited list of panel names to look for.

For example:

```
VBZS_FrontTabPanels (
    "Details¶Address"
)
```

If the currently visible panels are:

```
Customer
Details
Address
```

the result is:

```
Details
Address
```

`Customer` is visible, but it is not returned because it was not included in `param`.

Parameter order

The order of names in `param` does not control the order of the result.

For example:

```
VBZS_FrontTabPanels (
    "Address¶Customer"
)
```

may return:

```
Customer
Address
```

if `Customer` is an outer visible panel and `Address` is a visible panel nested inside it.

The parameter determines which visible panels may be returned. It does not sort them.

Similarity to FilterValues

When `param` is supplied, its behaviour is conceptually similar to FileMaker's native `FilterValues` function.

First, `VBZS_FrontTabPanels` determines which named tab and slide panels are currently visible.

The supplied list then acts as a filter on that result.

Conceptually:

```
Visible panels
      ↓
Filter using param
      ↓
Matching visible panels
```

This means `param` answers the question:

```
Which of these panels are currently visible?
```

rather than:

```
Return these panels in this order.
```

Example: testing selected panels

Suppose the visible structure is:

```
Main
└── Customer
    └── Contact
```

Calling:

```
VBZS_FrontTabPanels (
    "Contact¶Orders¶Main"
)
```

returns:

```
Main
Contact
```

if both `Main` and `Contact` are currently visible.

`Orders` is not returned because it is not currently visible.

The result follows the visible layout structure, not the order of the parameter list.

Using the function as a visibility test

Because `param` filters the visible panel list, the function can be used to test only the panels that are relevant to a particular calculation or script.

For example:

```
VBZS_FrontTabPanels (
    "Customer¶Supplier"
)
```

can be used when a calculation only needs to know whether the visible interface currently contains the `Customer` or `Supplier` panel.

There is no need for the calculation to inspect or process unrelated visible panels.

Notes

- Both tab panels and slide panels are supported.
- Panels must have an object name assigned in FileMaker's Inspector.
- Without `param`, all currently visible named panels are returned.
- With `param`, only matching visible panels are returned.
- `param` is a return-delimited list of object names.
- The order of `param` does not determine the result order.
- Nested visible panels are supported.
- Nested panels are returned from outermost to innermost.
- A visible outer panel is not automatically included when `param` is used. Its name must also occur in `param`.
- The result is a return-delimited list.

Design philosophy

FileMaker layouts can contain tab controls and slide controls nested inside other panel objects. Determining which combination of panels is currently visible can therefore become considerably more complicated than simply identifying one active tab.

`VBZS_FrontTabPanels` reduces that state to a simple FileMaker value list.

Without a parameter, the function answers:

```
Which named tab and slide panels are visible right now?
```

With a parameter, it answers:

```
Which of the panels I am interested in are visible right now?
```

The optional filtering behaviour follows the familiar principle of FileMaker's `FilterValues` function: the parameter restricts the values that may be returned without defining their order.

This allows scripts and calculations to respond to the current visible layout state without having to reproduce the complete panel structure in FileMaker logic.

VBZS_JSONParameters

Description

`VBZS_JSONParameters` creates FileMaker variables from the contents of a JSON object or JSON array.

The function is intended as a structured alternative to constructions where `Evaluate` and `Let` are used to create variables from calculation text such as:

```
$var1 = 15;
$var2 = "Some value";
```

Instead of constructing FileMaker calculation code, structured data can be passed as JSON. `VBZS_JSONParameters` then creates the corresponding FileMaker variables when the data is needed.

The function supports:

- root-level JSON objects;
- root-level JSON arrays;
- local FileMaker variables;
- optional namespaces;
- FileMaker repeating variables when processing arrays;
- global variables when explicitly requested.

Syntax

```
VBZS_JSONParameters ( json ; i ; namespace )
```

Parameters

`json`

A valid JSON object or JSON array to process.

For a JSON object, the keys at the supplied level are converted to FileMaker variable names and the corresponding JSON values are assigned to those variables.

A root-level JSON array receives special processing and can be converted into indexed variables and FileMaker repeating variables.

`i`

Reserved for future use.

This parameter must currently be left empty:

```
""
```

`namespace`

An optional prefix added to the FileMaker variables created by the function.

For example:

```
customer_
```

combined with the JSON keys:

```
id
name
```

creates:

```
$customer_id
$customer_name
```

Namespaces are particularly useful in larger scripts where several JSON objects may contain identical key names.

Result

The primary result of `VBZS_JSONParameters` is the set of FileMaker variables it creates.

When processing completes successfully, the function itself returns an empty result.

If an error occurs, the function returns:

```
?
```

For example, invalid JSON causes the function to return `?`.

This follows the familiar FileMaker convention of returning `?` when a calculation cannot produce a valid result.

Processing a JSON object

Given:

```
{
    "id": 5678,
    "name": "Acme BV"
}
```

the following calculation:

```
VBZS_JSONParameters (
    json ;
    "" ;
    ""
)
```

creates:

```
$id   = 5678
$name = Acme BV
```

The JSON keys become FileMaker variable names.

Using namespaces

The same JSON can be loaded using:

```
VBZS_JSONParameters (
    json ;
    "" ;
    "customer_"
)
```

which creates:

```
$customer_id   = 5678
$customer_name = Acme BV
```

A namespace is especially useful when multiple JSON structures are processed within the same script or script chain.

For example, an order and a customer may both contain:

```
id
name
```

Without namespaces, one set of variables could overwrite the other.

Using:

```
customer_
order_
```

produces separate variables such as:

```
$customer_id
$customer_name

$order_id
$order_name
```

Variable scope

A normal namespace creates local FileMaker variables.

For example:

```
namespace = "customer_"
```

creates:

```
$customer_id
$customer_name
```

Global variables can also be created when required.

Prefix the namespace itself with `$`:

```
namespace = "$customer_"
```

and the resulting variables are global FileMaker variables instead of local variables.

This allows the same namespacing mechanism to be used for either variable scope.

Nested objects and arrays

Nested objects and arrays encountered while processing a normal JSON object are deliberately not automatically expanded.

Consider:

```
{
    "id": 1234,
    "customer": {
        "id": 5678,
        "name": "Acme BV"
    }
}
```

Automatically traversing the complete JSON structure could result in several objects attempting to create:

```
$id
```

There would be no reliable way to determine which value should own that variable without automatically inventing namespaces.

If a contained object or array needs to be converted to variables, pass that object or array separately to `VBZS_JSONParameters`, preferably using an appropriate namespace.

Once an array itself is supplied as the root value, the function applies its special array processing as described below.

This keeps variable scope and naming under the control of the FileMaker developer.

Root-level JSON arrays

A root-level JSON array is a deliberate exception to the non-recursive behaviour.

Arrays receive special processing because they commonly represent lists of similar objects, such as order lines returned by an API.

Given:

```
[
    {
        "test": "value"
    },
    {
        "test": "value #2"
    },
    {
        "test": "value #3"
    }
]
```

and:

```
namespace = "demo_"
```

the array is first split into individual variables.

The complete JSON objects are stored using zero-based array indexes:

```
$demo_0 = {"test":"value"}
$demo_1 = {"test":"value #2"}
$demo_2 = {"test":"value #3"}
```

Each array element is subsequently processed as a variable list.

The `test` values therefore become a FileMaker repeating variable:

```
$demo_test
$demo_test[2]
$demo_test[3]
```

containing:

```
value
value #2
value #3
```

This recursive processing is specific to the handling of a root-level array. It does not cause arbitrary nested objects and arrays to be recursively traversed.

Array indexing and FileMaker repetitions

There are two different numbering conventions involved when processing an array.

The complete JSON array elements use JSON-style zero-based numbering:

```
$demo_0
$demo_1
$demo_2
```

The extracted values use FileMaker repeating variables:

```
$demo_test
$demo_test[2]
$demo_test[3]
```

The first repetition in FileMaker can be referenced without explicitly specifying `[1]`.

The first representation therefore corresponds to the original JSON array indexing, while the second follows FileMaker's normal repetition behaviour.

`max_repetition`

Whenever a JSON array is processed, `VBZS_JSONParameters` creates:

```
$max_repetition
```

This variable contains the number of objects in the root-level array that was just processed.

If a namespace is supplied, a namespaced version is created as well.

For example:

```
namespace = "demo_"
```

with three array objects creates:

```
$max_repetition      = 3
$demo_max_repetition = 3
```

The non-namespaced `$max_repetition` is always created deliberately.

This allows a script to process the array immediately after calling `VBZS_JSONParameters` without needing to take the namespace into account when determining the number of array objects.

The namespaced version remains available as part of the particular dataset.

Processing variable-length arrays

`$max_repetition` is particularly useful when processing data returned by an external API.

An order, for example, might contain only one order line or a large number of order lines.

After loading the JSON:

```
VBZS_JSONParameters (
    orderLines ;
    "" ;
    "order_"
)
```

the generated variables may resemble:

```
$order_article
$order_article[2]
$order_article[3]

$order_quantity
$order_quantity[2]
$order_quantity[3]

$order_price
$order_price[2]
$order_price[3]
```

while:

```
$max_repetition
```

contains the number of objects in the current JSON array.

A script can therefore process repetitions:

```
1 ... $max_repetition
```

without having to know the size of the order beforehand.

Avoiding artifacts from earlier arrays

`$max_repetition` also defines how much of the repeating variable structure belongs to the array that was most recently processed.

This is important because variables or repetitions created during an earlier operation may still exist in the current script context.

For example, processing an array with ten elements may create repetitions through:

```
$order_article[10]
```

If another array containing only three objects is subsequently loaded, values from the earlier operation may still exist at higher repetition numbers.

The newly created:

```
$max_repetition = 3
```

indicates that only repetitions 1 through 3 belong to the current array.

> **Important:** When processing an array, use `$max_repetition` to determine how many objects and corresponding repetitions belong to the current result.

Passing structured data between scripts

JSON provides a convenient way to pass structured data between FileMaker scripts without requiring global variables as the transport mechanism.

A script can receive JSON through:

```
Get ( ScriptParameter )
```

and convert it into appropriately namespaced variables:

```
VBZS_JSONParameters (
    Get ( ScriptParameter ) ;
    "" ;
    "order_"
)
```

This may create:

```
$order_id
$order_customer
$order_total
```

A subscript can return another JSON object, after which the calling script can process the returned structure using:

```
VBZS_JSONParameters (
    Get ( ScriptResult ) ;
    "" ;
    "result_"
)
```

This allows structured data to be transferred between scripts as JSON and converted to local FileMaker variables only where those variables are needed.

Converting JSON keys to FileMaker variable names

JSON key names may contain characters that are unsuitable for FileMaker variable names.

`VBZS_JSONParameters` therefore sanitizes the keys before creating variables.

The following characters are converted to an underscore:

```
, + - * / \ ^ & = ≠ > < [ ] { } ; : $
```

Multiple consecutive underscores are reduced to a single underscore.

For example, a conversion that would otherwise produce:

```
name___value
```

is normalized to:

```
name_value
```

Parentheses and repetitions

Parentheses receive special treatment.

```
(  →  .
)  →  .
```

This is particularly relevant when processing JSON originating from the FileMaker Data API, where parentheses can indicate repetitions.

Additional normalization is performed on combinations that may already exist in a JSON key or may be created during key conversion.

The following combinations:

```
_.
._
..
```

are normalized to:

```
.
```

This prevents intermediate conversions from leaving unnecessary combinations of underscores and periods in the resulting FileMaker variable name.

Maximum root-level elements

The current implementation supports a maximum of:

```
999
```

elements at the root level.

For a JSON object, this means a maximum of 999 root-level keys.

For a root-level JSON array, this means a maximum of 999 objects.

If the supplied JSON exceeds this limit, the function does not process it.

> **Important:** `VBZS_JSONParameters` currently supports a maximum of 999 root-level elements.

Nested structures do not increase the root-level key count of an ordinary JSON object because they are not automatically expanded.

Error handling

A successful call returns an empty result:

```
""
```

If processing fails, the function returns:

```
?
```

Invalid JSON is one example of input that results in `?`.

The result can therefore be checked when a script needs to determine whether the supplied JSON was successfully processed.

For example:

```
Set Variable [
    $result ;
    Value: VBZS_JSONParameters (
        Get ( ScriptParameter ) ;
        "" ;
        "order_"
    )
]

If [ $result = "?" ]
    # JSON could not be processed
End If
```

Notes

- The supplied JSON must be a valid JSON object or JSON array.
- The `i` parameter is reserved and must currently be left empty.
- A successful call returns an empty result.
- An error returns `?`.
- JSON keys are converted into FileMaker variable names.
- Namespaces help prevent variables from unrelated JSON structures from overwriting each other.
- Namespaces normally create local `$variables`.
- Starting the namespace with `$` creates global `$$variables`.

- Nested objects and arrays encountered in a normal JSON object are not automatically expanded.
- A separately supplied root-level array receives special recursive processing.
- Root-array elements are made available as zero-based indexed variables.
- Keys within those array elements are loaded as FileMaker repeating variables.
- `$max_repetition` is always created when an array is processed.
- `$max_repetition` contains the number of objects in the root-level array that was just processed.
- A namespaced `max_repetition` is additionally created when a namespace is used.
- A maximum of 999 root-level elements can be processed.

Design philosophy

`VBZS_JSONParameters` is designed to make structured parameter passing practical in FileMaker.

Instead of building strings containing FileMaker variable assignments and evaluating them using `Evaluate` and `Let`, data can remain JSON while it is passed between scripts and converted to variables only when those variables are actually needed.

Namespaces make it possible to work with several JSON datasets containing common key names without constantly risking accidental variable collisions.

Nested JSON structures encountered while processing a normal JSON object are processed explicitly rather than recursively. This prevents the function from inventing namespaces, overwriting variables unexpectedly or recursively traversing structures whose meaning is known much better by the FileMaker developer than by the function.

Root-level arrays are treated differently because they represent a common and predictable structure: a variable-length collection of similar objects.

Such arrays are split into indexed JSON objects and those objects are subsequently processed into FileMaker repeating variables. `$max_repetition` contains the number of objects in the most recently loaded root-level array and therefore determines the relevant range of repetitions.

This makes patterns such as processing webshop order lines straightforward even when the number of items is not known beforehand.

The central principle is:

Use JSON to transport structured data, namespaces to keep datasets separate, and FileMaker variables only where the data is actually needed.

VBZS_ListLogic

Description

`VBZS_ListLogic` compares two FileMaker value lists using one of three logical operations: `AND`, `XOR` or `NOT`.

The operations follow the familiar principles of Boolean logic, applied to the individual values in return-delimited FileMaker lists.

Comparisons are case-insensitive, but values returned by the function retain their original capitalization.

The function processes values from top to bottom, starting with `lista`.

Syntax

```
VBZS_ListLogic ( type ; lista ; listb )
```

Parameters

`type`

Defines the logical operation to perform.

The following operations are supported:

Type	Result
`AND`	Values from `lista` that also occur in `listb`
`XOR`	Values that occur exclusively in either `lista` or `listb`
`NOT`	Values from `lista` that do not occur in `listb`

`lista`

The primary value list.

Values in `lista` are processed from top to bottom.

The original value from `lista`, including its capitalization, is retained when that value qualifies for the result.

Duplicates in `lista` are preserved when they satisfy the selected logical operation.

`listb`

The value list against which `lista` is compared.

Comparisons are case-insensitive.

Duplicates in `listb` do not increase the number of matches produced from `lista`. For matching purposes, the important question is whether a value occurs in `listb`, not how many times it occurs.

Result

Returns a return-delimited FileMaker value list containing the values that satisfy the selected logical operation.

The function processes the lists from left to right and the values within each list from top to bottom.

The original capitalization of a returned value is preserved.

Case-insensitive comparison

All comparisons are case-insensitive.

For example:

```
AbC
abc
ABC
aBc
```

are considered identical when determining whether values match.

However, the function does not change the original capitalization of the value included in the result.

For example:

```
VBZS_ListLogic (
    "AND" ;
    "AbC" ;
    "aBc"
)
```

returns:

```
AbC
```

The values match because the comparison is case-insensitive, but `AbC` is returned because that is the value encountered in `lista`.

AND

`AND` returns values from `lista` that also occur in `listb`.

For example:

```
VBZS_ListLogic (
    "AND" ;
    "A¶B¶C" ;
    "B¶C¶D"
)
```

returns:

```
B
C
```

Duplicates with AND

Qualifying duplicates in `lista` are preserved.

For example:

```
VBZS_ListLogic (
    "AND" ;
    "A¶A" ;
    "C¶B¶a"
)
```

returns:

```
A
A
```

Both values in `lista` match `a` in `listb`.

The comparison is case-insensitive, so `A` and `a` are considered equal.

The original `A` values from `lista` are returned.

Duplicates in `listb` do not create additional results.

For example:

```
VBZS_ListLogic (
    "AND" ;
    "A¶A" ;
    "A¶A¶A"
)
```

returns:

```
A
A
```

The three occurrences of `A` in `listb` do not produce three matches. `lista` remains the primary list and contains two qualifying values.

XOR

`XOR` returns values that occur exclusively in one of the two lists.

Values occurring in both lists are excluded.

For example:

```
VBZS_ListLogic (
    "XOR" ;
    "A¶B" ;
    "B¶C"
)
```

returns:

```
A
C
```

`B` occurs in both lists and is therefore excluded.

Values exclusive to `lista` are processed first, followed by qualifying values from `listb`.

For example:

```
VBZS_ListLogic (
    "XOR" ;
    "A¶A" ;
    "C¶B"
)
```

returns:

```
A
A
C
B
```

The duplicate `A` values from `lista` are preserved.

NOT

`NOT` returns values from `lista` that do not occur in `listb`.

Unlike `AND` and `XOR`, the direction of the comparison is fundamental to the operation.

For example:

```
VBZS_ListLogic (
    "NOT" ;
    "A¶B¶C" ;
    "B¶C¶D"
)
```

returns:

```
A
```

Reversing the lists performs a different comparison:

```
VBZS_ListLogic (
    "NOT" ;
    "B¶C¶D" ;
    "A¶B¶C"
)
```

returns:

```
D
```

Duplicates with NOT

Duplicates from `lista` are preserved when they satisfy the condition.

For example:

```
VBZS_ListLogic (
    "NOT" ;
    "A¶A" ;
    "C¶B"
)
```

returns:

```
A
A
```

Both occurrences of `A` in `lista` are absent from `listb` and are therefore returned.

List order

The function processes its input from left to right and top to bottom.

`lista` is processed first.

This means that, although `AND` and `XOR` are logically symmetrical operations, exchanging `lista` and `listb` can affect the order and capitalization of the returned values.

For example:

```
VBZS_ListLogic (
    "AND" ;
    "AbC" ;
    "aBc"
)
```

returns:

```
AbC
```

while:

```
VBZS_ListLogic (
    "AND" ;
    "aBc" ;
    "AbC"
)
```

returns:

```
aBc
```

The logical match is identical, but the returned representation follows the value encountered in `lista`.

Logical overview

Given two lists A and B, the three operations can be summarized as:

```
AND : values in A that are also in B
XOR : values in A or B, but not in both
NOT : values in A that are not in B
```

The operations follow these logical relationships:

```
AND ( A ; B ) = AND ( B ; A )
XOR ( A ; B ) = XOR ( B ; A )
NOT ( A ; B ) ≠ NOT ( B ; A )
```

For `AND` and `XOR`, this equivalence describes which values logically qualify. Because the function preserves input order, duplicates and original capitalization, exchanging the actual lists may produce a differently ordered or formatted result.

Notes

- `lista` and `listb` are return-delimited FileMaker value lists.
- Comparisons are case-insensitive.
- Returned values retain their original capitalization.
- Values are processed from top to bottom.
- `lista` is processed before `listb`.
- Qualifying duplicates from `lista` are preserved.
- Duplicates in `listb` do not increase the number of matches produced from `lista`.
- `AND` returns values from `lista` that also occur in `listb`.
- `NOT` returns values from `lista` that do not occur in `listb`.
- `XOR` returns values occurring exclusively in either list.
- With `XOR`, qualifying values from `lista` appear before qualifying values from `listb`.

Design philosophy

`VBZS_ListLogic` applies simple logical operations directly to FileMaker value lists.

The function deliberately works with the lists as supplied rather than first converting them into unique sets. This means that the original order, capitalization and qualifying duplicate values from `lista` can remain meaningful.

`lista` acts as the primary list and is processed from top to bottom. `listb` provides the comparison used to determine whether each value satisfies the selected logical operation.

This makes `VBZS_ListLogic` useful when the values themselves are important, but their original representation and list order should not be discarded merely because a logical comparison is being performed.

VBZS_Local2UTC

Description

`VBZS_Local2UTC` converts or formats a local FileMaker timestamp for use in different timestamp representations, including ISO 8601 and UTC/Zulu time.

The function takes a local timestamp together with its normal GMT timezone offset. Daylight Saving Time is calculated automatically using the built-in Western European DST rules.

Depending on the requested output format, the function can:

- return the timestamp using the system format;
- format the timestamp for data entry;
- include the current GMT offset;
- return a full ISO 8601 representation;
- convert the local timestamp to UTC/Zulu time.

Syntax

```
VBZS_Local2UTC ( LocalTime ; TimeZone ; iso8601 )
```

Parameters

`LocalTime`

The local FileMaker timestamp to convert or format.

`LocalTime` should be supplied as a FileMaker timestamp.

For example:

```
TimeStamp (
    Date ( 10 ; 5 ; 2026 ) ;
    Time ( 15 ; 0 ; 0 )
)
```

`TimeZone`

The normal GMT offset of the supplied local timestamp, expressed in hours.

For example:

```
1
```

represents GMT+1 during standard time.

Fractional timezone offsets are supported. This allows timezones with half-hour offsets to be represented, for example:

```
5.5
```

Daylight Saving Time is handled separately by the function and may add another hour to the effective offset.

`iso8601`

Determines the representation of the returned timestamp.

The following values are supported:

Value	Format
empty or `0`	Timestamp using the system format
`1`	Local timestamp with the current GMT offset
`A`	ANSI data-entry format
`D`	Database data-entry format
`F`	Full ISO 8601 representation including GMT offset
`Z`	UTC/Zulu representation

Daylight Saving Time

`VBZS_Local2UTC` contains built-in Daylight Saving Time rules for Western Europe.

The supplied `TimeZone` represents the normal GMT offset. During Daylight Saving Time, the function adds one additional hour to that offset.

For example:

```
TimeZone = 1
```

represents:

```
UTC+01:00
```

during standard time and:

```
UTC+02:00
```

during Daylight Saving Time.

The built-in transitions are:

- Daylight Saving Time starts on the last Sunday of March at `00:00 UTC`.
- Daylight Saving Time ends on the last Sunday of October at `01:00 UTC`.

> **Important:** The built-in Daylight Saving Time calculation follows Western European rules. `VBZS_Local2UTC` should not be used to convert local timestamps from regions that use different DST rules, such as the United States.

Standard format

When `iso8601` is empty or `0`, the timestamp is returned using the system timestamp format.

```
VBZS_Local2UTC (
    LocalTime ;
    1 ;
    ""
)
```

The actual representation therefore depends on the system settings of the computer on which FileMaker performs the calculation.

Format `1`

Using:

```
iso8601 = 1
```

returns the local timestamp together with the GMT offset that applies at that moment.

For example:

```
VBZS_Local2UTC (
    TimeStamp (
        Date ( 10 ; 5 ; 2026 ) ;
        Time ( 15 ; 0 ; 0 )
    ) ;
    1 ;
    1
)
```

returns:

```
2026-10-05 15:00:00+02:00
```

The normal timezone is GMT+1, but October 5 falls within the built-in Daylight Saving Time period. The applicable offset is therefore `+02:00`.

The local clock time itself remains:

```
15:00:00
```

ANSI format

Using:

```
iso8601 = "A"
```

returns the timestamp using an ANSI-style data-entry format:

```
MM/DD/YYYY hh:mm:ss
```

For example:

```
10/05/2026 15:00:00
```

This format can be useful when supplying timestamp data to the FileMaker Data API when an explicit `dateformats` option has not been specified.

The date is deliberately represented using the month/day/year order expected by that data-entry format.

Database format

Using:

```
iso8601 = "D"
```

returns the timestamp in database data-entry format:

```
YYYY+MM+DD hh:mm:ss
```

For example:

```
2026+10+05 15:00:00
```

This representation is also intended for data exchange such as FileMaker Data API operations.

The year-month-day representation avoids ambiguity introduced by regional date formats. It can therefore be interpreted consistently without depending on whether the receiving database or computer uses a day/month/year or month/day/year locale.

Full ISO 8601 format

Using:

```
iso8601 = "F"
```

returns the local timestamp together with the GMT offset applying at that moment.

The format is:

```
YYYY-MM-DD hh:mm:ss+hh:mm
```

For example:

```
VBZS_Local2UTC (
    TimeStamp (
        Date ( 10 ; 5 ; 2026 ) ;
        Time ( 15 ; 0 ; 0 )
    ) ;
    1 ;
    "F"
)
```

returns:

```
2026-10-05 15:00:00+02:00
```

The timestamp continues to represent the local clock time.

The GMT offset identifies the relationship between that local time and UTC.

UTC/Zulu format

Using:

```
iso8601 = "Z"
```

converts the supplied local timestamp to UTC.

The format is:

```
YYYY-MM-DDThh:mm:ssZ
```

For example:

```
VBZS_Local2UTC (
    TimeStamp (
        Date ( 10 ; 5 ; 2026 ) ;
        Time ( 15 ; 0 ; 0 )
    ) ;
    1 ;
    "Z"
)
```

returns:

```
2026-10-05T13:00:00Z
```

At this date, the local GMT+1 timezone is using Daylight Saving Time, making the effective offset:

```
+02:00
```

The local time:

```
15:00:00
```

is therefore converted to:

```
13:00:00Z
```

`F` versus `Z`

The distinction between `F` and `Z` is important.

`F` represents the local timestamp and includes its current offset from GMT:

```
2026-10-05 15:00:00+02:00
```

`Z` represents the same moment converted to UTC:

```
2026-10-05T13:00:00Z
```

Both values therefore describe the same moment in time, but from different timezone perspectives.

Fractional timezone offsets

`TimeZone` is not limited to whole hours.

For regions with a half-hour offset, a fractional value can be supplied.

For example:

```
5.5
```

represents an offset of five hours and thirty minutes before any applicable built-in DST correction.

Keep in mind that the automatic DST calculation still follows the Western European DST rules.

A fractional `TimeZone` value therefore does not automatically make the function appropriate for a region whose Daylight Saving Time rules differ from Western Europe.

Regional limitation

The timezone itself is supplied by the caller, but the Daylight Saving Time rules are built into the function.

This distinction is important.

Changing:

```
TimeZone
```

changes the basic GMT offset, but it does not change the rules used to determine when Daylight Saving Time starts and ends.

Consequently, supplying the GMT offset of another region is only appropriate when that region follows the same DST transition rules.

`VBZS_Local2UTC` is primarily intended for timestamps using Western European DST rules.

Notes

- `LocalTime` should be supplied as a FileMaker timestamp.
- `TimeZone` specifies the normal GMT offset in hours.
- Fractional timezone offsets are supported.
- Daylight Saving Time is calculated automatically.
- The DST rules are fixed to the Western European transition dates and times.
- Empty or `0` uses the system timestamp representation.
- `1` returns local time with the applicable GMT offset.
- `A` produces `MM/DD/YYYY hh:mm:ss`.
- `D` produces `YYYY+MM+DD hh:mm:ss`.
- `F` produces local time in full ISO 8601 form with its GMT offset.
- `Z` converts the timestamp to UTC/Zulu time.

- `F` retains the local clock time.
- `Z` changes the clock time to its UTC equivalent.
- Fractional seconds are not part of the documented output format.

Design philosophy

`VBZS_Local2UTC` is intended for situations where a FileMaker timestamp needs to be exchanged with other systems in a predictable representation.

A FileMaker timestamp by itself represents a date and time but does not necessarily communicate the relationship of that local time to UTC. This becomes important when timestamps are sent through APIs or exchanged between systems in different locations.

`VBZS_Local2UTC` combines three pieces of information:

```
local timestamp
+
normal GMT offset
+
Western European DST rules
```

and can represent the result in several formats suitable for display, data entry or data exchange.

The `F` and `Z` formats address two different needs.

Use `F` when the local timestamp and its UTC offset should both remain visible.

Use `Z` when the moment should be converted to an unambiguous UTC/Zulu timestamp.

The `A` and `D` formats provide alternative data-entry representations, with `D` particularly useful when the date should be interpreted independently of regional date settings.

VBZS_ModKeys

Description

`VBZS_ModKeys` reads the modifier keys reported by FileMaker and stores the state of each supported key in a separate FileMaker variable.

Each variable contains:

```
0
```

when the modifier key is not active, or:

```
1
```

when the modifier key is active at the moment the function is evaluated.

The function is based on FileMaker's native:

```
Get ( ModifierKeys )
```

Instead of returning a single combined bit value, `VBZS_ModKeys` provides the state of each modifier key as an individual variable.

Syntax

```
VBZS_ModKeys ( prefix )
```

Parameters

`prefix`

An optional namespace added before the name of each variable created by the function.

If `prefix` is empty, the function creates:

```
$apple
$alt
$ctrl
$caps
$shift
```

If a namespace is supplied, it is inserted before each variable name.

For example:

```
VBZS_ModKeys ( "mod_" )
```

creates:

```
$mod_apple
$mod_alt
$mod_ctrl
$mod_caps
$mod_shift
```

Result

`VBZS_ModKeys` does not return a calculation result.

Its only result is the set of FileMaker variables created when the function is evaluated.

Created variables

Without a namespace, the following variables are created:

Variable	Modifier key
`$apple`	Apple Command key
`$alt`	Alt or Option key
`$ctrl`	Control key
`$caps`	Caps Lock
`$shift`	Shift key

Each variable contains either `0` or `1`.

Multiple variables can contain `1` at the same time when several modifier keys are active simultaneously.

Example without a namespace

```
VBZS_ModKeys ( "" )
```

creates:

```
$apple
$alt
$ctrl
$caps
$shift
```

For example, if Shift and Alt are active when the function is evaluated, the variables may contain:

```
$apple = 0
$alt   = 1
$ctrl  = 0
$caps  = 0
$shift = 1
```

The function itself returns an empty result.

Example with a namespace

```
VBZS_ModKeys ( "keyboard_" )
```

creates:

```
$keyboard_apple
$keyboard_alt
$keyboard_ctrl
$keyboard_caps
$keyboard_shift
```

The namespace can help prevent the generated variables from conflicting with variables already used by a script.

Global variables

A namespace beginning with `$` creates global FileMaker variables instead of local variables.

For example:

```
VBZS_ModKeys ( "$keyboard_" )
```

creates:

```
$$keyboard_apple
$$keyboard_alt
$$keyboard_ctrl
$$keyboard_caps
$$keyboard_shift
```

Without the additional `$` in the namespace, normal local FileMaker variables are created.

Platform behaviour

`VBZS_ModKeys` creates the same set of variables on macOS and Windows.

On macOS, `$apple` represents the Apple Command key as reported by FileMaker's `Get ( ModifierKeys )`.

On Windows, `$apple` is still created, but its value is always:

```
0
```

FileMaker's `Get ( ModifierKeys )` does not provide an equivalent value for the Windows key. Consequently, `VBZS_ModKeys` does not create a separate variable for the Windows key.

Creating `$apple` on both platforms keeps the set of generated variables consistent between macOS and Windows.

Evaluation time

The variables reflect the modifier-key state at the moment `VBZS_ModKeys` is evaluated.

If the state of the keyboard changes afterward, the generated variables are not automatically updated.

Evaluate the function again whenever the current modifier-key state is required.

Notes

- The function is based on FileMaker's `Get ( ModifierKeys )`.
- The combined modifier-key value is split into separate variables.
- Every generated variable contains either `0` or `1`.
- Several modifier-key variables can contain `1` simultaneously.
- `prefix` is optional.
- An empty `prefix` creates `$apple`, `$alt`, `$ctrl`, `$caps` and `$shift`.
- A normal prefix acts as a namespace for local variables.
- A prefix beginning with `$` creates global variables.
- `$apple` is created on both macOS and Windows.
- On Windows, `$apple` is always `0`.
- No variable is created for the Windows key.
- The function itself returns an empty result.

Design philosophy

FileMaker's native `Get ( ModifierKeys )` returns a combined numeric value representing the active modifier keys.

Using that result normally requires the developer to interpret the individual bit values before a script or calculation can determine which keys are active.

`VBZS_ModKeys` performs that interpretation once and exposes the individual states as simple FileMaker variables.

This allows a script to use variables such as:

```
$shift
$alt
$ctrl
```

without repeatedly decoding the combined value returned by `Get ( ModifierKeys )`.

The optional namespace makes the generated variables suitable for use in larger scripts, while maintaining a consistent variable set across macOS and Windows.

VBZS_OntDiakriet

Description

`VBZS_OntDiakriet` converts supported accented and special characters to predefined ASCII equivalents.

The function is intended for situations where text must be supplied to web services or other external systems that do not reliably accept diacritics or certain extended characters.

`VBZS_OntDiakriet` operates as a character filter with replacement:

- supported normal characters are preserved;
- supported characters with diacritics are replaced by their ASCII equivalent;
- a small number of special characters are replaced by two ASCII characters;
- characters for which no supported conversion exists are removed.

Syntax

```
VBZS_OntDiakriet ( text )
```

Parameters

`text`

The text to be converted.

Result

Returns the filtered text with supported diacritics and special characters replaced by predefined ASCII equivalents.

Characters that cannot be preserved or converted by the function are removed from the result.

Example

```
VBZS_OntDiakriet ( "Üßéř" )
```

returns:

```
Uszer
```

In this example the conversions include:

```
Ü  →  U
ß  →  sz
é  →  e
ř  →  r
```

Diacritic replacement

Supported Latin characters containing accents, umlauts, cedillas, carons and similar diacritics are converted to their corresponding character without the diacritic.

Examples include conversions such as:

```
á  →  a
é  →  e
Ü  →  U
ç  →  c
ř  →  r
```

Capitalization is preserved where an equivalent capital ASCII character is available.

The purpose is to convert the actual text rather than merely perform a diacritic-insensitive comparison.

Multi-character replacements

A limited number of special characters are converted to two ASCII characters instead of a single character.

These include:

```
ß
æ
œ
ff
fi
fl
ffi
ffl
ſt
lj
nj
Þ
Ĳ
```

and their capital variants where such variants exist.

These characters require a predefined multi-character replacement because simply removing a diacritic would not produce a useful ASCII representation.

For example:

```
ß  →  sz
```

as demonstrated by:

```
Üßéř  →  Uszer
```

Unsupported characters

`VBZS_OntDiakriet` is not a general-purpose Unicode transliteration function.

Characters for which the function has no supported conversion are removed.

This means that the function should be considered a filter with replacement rather than a function that attempts to find an approximate ASCII representation for every possible Unicode character.

Conceptually:

```
Supported normal character
    → keep

Supported character with diacritic
    → replace

Supported special character
    → replace with predefined ASCII representation

Unsupported character
    → remove
```

FileMaker searches

FileMaker itself already performs many text searches without distinguishing between common diacritic variants.

For example, a FileMaker search for:

```
uber
```

can also find:

```
über
```

and vice versa.

`VBZS_OntDiakriet` is therefore not primarily intended to make FileMaker searches diacritic-insensitive.

The function is useful when the text itself must actually be converted before it is stored, exported or supplied to another system.

Use with external systems

Some external systems and web services accept only a limited character set or behave unpredictably when supplied with accented or other extended characters.

In such situations:

```
VBZS_OntDiakriet ( text )
```

can be used to create a restricted representation of the text before it is sent to the external system.

Because unsupported characters are removed, the returned value should be considered converted text rather than a lossless representation of the original value.

Notes

- The function returns text.
- Supported diacritics are replaced by predefined ASCII equivalents.
- Capitalization is preserved where applicable.
- Some special characters are converted to two ASCII characters.
- Unsupported characters are removed.
- The function is a filter with replacement.
- The function is not a general Unicode transliteration mechanism.
- The conversion changes the actual text.
- FileMaker’s own search behaviour may already be sufficient when only diacritic-insensitive searching is required.

Design philosophy

`VBZS_OntDiakriet` was created for situations where text needs to be supplied to systems that cannot reliably handle diacritics or certain extended characters.

Its purpose is deliberately limited.

Rather than trying to understand arbitrary Unicode characters or perform language-dependent transliteration, the function uses a known set of conversions.

Characters with a supported equivalent are converted. Characters that cannot be handled by the defined character set are removed.

This makes the behaviour predictable when generating text for systems that require a restricted character set.

VBZS_QualifiedEmailAddresses

Syntax

```
VBZS_QualifiedEmailAddresses ( ListOfEmailAddresses { ; CharacterSet } )
```

Description

`VBZS_QualifiedEmailAddresses` validates one or more email addresses and returns only the addresses that meet the validation rules.

The function is intended for practical validation of email addresses before they are stored, processed, or used for sending email.

Multiple email addresses may be supplied in a single parameter. The function recognizes several commonly used separators, allowing input copied from fields, email applications, spreadsheets, or manually entered lists to be processed directly.

The function does not correct invalid email addresses. Invalid addresses are simply omitted from the result.

Parameters

ListOfEmailAddresses

One or more email addresses.

Multiple addresses may be separated by:

- FileMaker returns
- line feeds
- tabs
- spaces
- commas
- semicolons

Multiple consecutive separators are treated as separators and do not generate empty values.

CharacterSet

Optional.

Specifies which alphabetic characters may occur in an email address.

Supported values are:

Value	Allowed alphabetic characters
empty or omitted	Latin characters including European diacritics
`Latin`	ASCII Latin letters `A-Z` and `a-z` only
`Greek`	Default character set plus Greek
`Cyrillic`	Default character set plus Cyrillic
`All`	Default character set plus Greek and Cyrillic

The value is not case-sensitive.

An empty or unrecognized value uses the default character set.

The `Latin` option can be used when compatibility with email systems that reject international or accented characters is required.

Result

The function returns a return-separated list containing only the valid email addresses.

Invalid addresses are omitted.

If no valid addresses are found, the function returns an empty result.

The original spelling and capitalization of accepted addresses are preserved.

Validation

Each email address is checked independently.

The following rules are applied:

- The address must contain exactly one `@`.
- The local part before `@` may not be empty.
- The domain after `@` may not be empty.
- The local part may contain no more than 64 characters.
- The complete email address may contain no more than 254 characters.
- Consecutive periods (`..`) are not allowed in the local part.
- The local part may not start or end with a period.
- The domain may not start or end with a period.
- The domain must contain at least one period.
- The final domain component must contain at least two characters.
- Domain labels may not start or end with a hyphen.
- The final domain component may contain alphabetic characters only.
- Characters not permitted by the selected character set are rejected.

The local part additionally supports the following characters:

```
.!#$%&'*+-/=?^_`{|}~
```

Digits `0-9` are permitted.

Domain names are limited to alphabetic characters, digits, periods and hyphens, subject to the selected character set.

Examples

Single email address

```
VBZS_QualifiedEmailAddresses ( "john@example.com" )
```

Returns:

```
john@example.com
```

Multiple addresses

```
VBZS_QualifiedEmailAddresses (
    "john@example.com; jane@example.com,info@example.nl"
)
```

Returns:

```
john@example.com
jane@example.com
info@example.nl
```

Semicolons and commas are automatically interpreted as separators.

Spaces, tabs and returns can also be used as separators.

Invalid addresses are removed

Given:

```
john@example.com
john..doe@example.com
john@example
john@exam_ple.com
jane@example.net
```

Using:

```
VBZS_QualifiedEmailAddresses ( addresses )
```

Returns:

```
john@example.com
jane@example.net
```

Invalid addresses do not generate an error and are not included in the result.

Character sets

Default

Without the optional `CharacterSet` parameter, European Latin characters with diacritics are accepted.

Given:

```
john@example.com
björn@example.com
søren@example.dk
```

Using:

```
VBZS_QualifiedEmailAddresses ( addresses )
```

Returns:

```
john@example.com
björn@example.com
søren@example.dk
```

Latin

Use `Latin` when accented or other non-ASCII alphabetic characters should not be accepted.

```
VBZS_QualifiedEmailAddresses ( addresses ; "Latin" )
```

For:

```
john@example.com
björn@example.com
søren@example.dk
```

Returns:

```
john@example.com
```

`Latin` permits only the alphabetic characters:

```
A-Z
a-z
```

This restriction applies to alphabetic characters only. Digits and permitted email punctuation remain available.

Greek

```
VBZS_QualifiedEmailAddresses ( addresses ; "Greek" )
```

The `Greek` option permits the default European Latin character set and Greek characters.

For example:

```
john@example.com
björn@example.com
δοκιμή@example.com
```

all three addresses are accepted.

Greek characters with accents and polytonic Greek characters are supported.

Cyrillic

```
VBZS_QualifiedEmailAddresses ( addresses ; "Cyrillic" )
```

The `Cyrillic` option permits the default European Latin character set and Cyrillic characters.

For example:

```
john@example.com
björn@example.com
тест@example.com
```

all three addresses are accepted.

All

```
VBZS_QualifiedEmailAddresses ( addresses ; "All" )
```

`All` permits:

- Latin
- European Latin characters with diacritics
- Greek
- Cyrillic

For example:

```
john@example.com
björn@example.com
søren@example.dk
δοκιμή@example.com
тест@example.com
```

all five addresses are accepted.

Input normalization

`VBZS_QualifiedEmailAddresses` is designed to handle lists entered by users.

For example:

```
john@example.com; jane@example.com, info@example.nl sales@example.net
```

is interpreted as four individual email addresses.

The recognized separators are logically treated as FileMaker value separators before the individual addresses are validated.

The function does not remove invalid characters in order to make an address valid.

For example:

```
john @example.com
```

is not silently changed into:

```
john@example.com
```

The function validates the supplied values rather than correcting them.

Notes

`VBZS_QualifiedEmailAddresses` is intended as a practical email-address validator.

Email standards permit constructs and international character combinations that are not necessarily accepted by every mail server, mail client, or receiving environment.

For this reason, the function deliberately applies a restricted and predictable set of validation rules rather than attempting to accept every theoretically valid email-address syntax.

The default character set supports European Latin characters with diacritics. The `Latin` option provides a more restrictive compatibility mode for environments where accented characters in email addresses are not accepted.

The `Greek`, `Cyrillic`, and `All` options should be used when the target email infrastructure is known to support these character sets.

The function validates the syntax and character usage of an email address. It does not verify whether:

- the domain exists;
- the domain has mail servers;
- the mailbox exists;
- the receiving server accepts the address;
- an email can actually be delivered.

See also

The output of `VBZS_QualifiedEmailAddresses` is a FileMaker return-separated value list and can therefore be used directly with FileMaker list and value functions such as `GetValue`, `ValueCount`, and `FilterValues`.

VBZS_ResponseCode

Description

`VBZS_ResponseCode` extracts the HTTP response code from a response header returned by FileMaker's `Insert from URL` using the cURL `-D` option.

The function creates two FileMaker variables:

```
$responseCode
$responseMessage
```

`$responseCode` contains the numeric HTTP response code.

`$responseMessage` contains the standard response message associated with that code.

For example, an HTTP response code of `200` results in:

```
$responseCode    = 200
$responseMessage = OK
```

The two variables have fixed names. `VBZS_ResponseCode` does not provide a namespace option.

Syntax

```
VBZS_ResponseCode ( header )
```

Parameters

`header`

The response header returned by an `Insert from URL` operation using the cURL `-D` option.

The function looks for an HTTP response beginning with:

```
HTTP/
```

followed by a numeric response code after the first space.

Result

When a valid and recognized HTTP response is found, the function creates:

```
$responseCode
$responseMessage
```

For example:

```
HTTP/1.1 200 OK
```

produces:

```
$responseCode    = 200
$responseMessage = OK
```

The response message is determined by `VBZS_ResponseCode` from its internal list of known HTTP response codes. It does not depend on the server supplying exactly the same response text in the header.

Multiple HTTP responses

A response header may contain more than one HTTP response.

This can occur, for example, when a request is redirected and the collected headers contain responses from more than one stage of the request.

When multiple responses beginning with:

```
HTTP/
```

are present, `VBZS_ResponseCode` processes the last response.

For example, a header containing responses such as:

```
HTTP/1.1 301 Moved Permanently
...

HTTP/1.1 200 OK
...
```

is processed as:

```
$responseCode    = 200
$responseMessage = OK
```

The final HTTP response is therefore the one represented by the generated variables.

Invalid response header

If the supplied value does not contain a valid HTTP response header, the function returns:

```
Missing HTTP response header
```

A header is considered invalid when the required HTTP response cannot be identified, for example when:

- no response starts with `HTTP/`;
- no numeric response code can be found after the first space.

When an invalid header is processed, both variables are cleared:

```
$responseCode    = ""
$responseMessage = ""
```

The variables are written on every call. Values left by an earlier call are therefore not retained when the current response header is invalid.

Unknown response code

If a valid HTTP response header is found but its numeric response code is not known by `VBZS_ResponseCode`, the function returns:

```
Undefined error
```

This distinguishes an unknown HTTP response code from a missing or malformed HTTP response header.

Supported response messages

`VBZS_ResponseCode` contains predefined messages for the following HTTP response codes.

Informational responses

Code	Message
100	Continue
101	Switching Protocols

Successful responses

Code	Message
200	OK

Code	Message
201	Created
202	Accepted
203	Non-Authoritative Information
204	No Content
205	Reset Content
206	Partial Content

Redirection responses

Code	Message
300	Multiple Choices
301	Moved Permanently
302	Found
303	See Other
304	Not Modified
305	Use Proxy
306	Unused
307	Temporary Redirect

Client error responses

Code	Message
400	Bad Request
401	Unauthorized
402	Payment Required
403	Forbidden
404	Not Found
405	Method Not Allowed
406	Not Acceptable
407	Proxy Authentication Required

Code	Message
408	Request Timeout
409	Conflict
410	Gone
411	Length Required
412	Precondition Failed
413	Request Entity Too Large
414	Request-URI Too Long
415	Unsupported Media Type
416	Requested Range Not Satisfiable
417	Expectation Failed

Server error responses

Code	Message
500	Internal Server Error
501	Not Implemented
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout
505	HTTP Version Not Supported

Example

After an `Insert from URL` operation using the cURL `-D` option, suppose `header` contains a response ending in:

```
HTTP/1.1 200 OK
Content-Type: application/json
...
```

The calculation:

```
VBZS_ResponseCode ( header )
```

creates:

```
$responseCode    = 200
$responseMessage = OK
```

These variables can subsequently be used directly by the FileMaker script.

For example:

```
If [ $responseCode = 200 ]
    # Process successful response
End If
```

Response messages

`$responseMessage` is based on the numeric response code rather than relying on the response message supplied by the remote server.

This provides a consistent message for the HTTP codes recognized by the function.

For example, response code:

```
404
```

produces:

```
$responseMessage = Not Found
```

and:

```
503
```

produces:

```
$responseMessage = Service Unavailable
```

Notes

- The function is intended for response headers obtained using the cURL `-D` option with FileMaker's `Insert from URL`.
- The response must contain a line beginning with `HTTP/`.
- The numeric response code is read from the HTTP response line.
- `$responseCode` contains the numeric HTTP response code.
- `$responseMessage` contains the predefined message corresponding to a recognized response code.
- The variables always have the names `$responseCode` and `$responseMessage`.
- Namespaces are not supported.

- When several HTTP responses are present, the last response beginning with `HTTP/` is used.
- Both variables are written on every invocation.
- An invalid response header clears both variables.
- An invalid or missing HTTP response returns `Missing HTTP response header`.
- An unknown response code returns `Undefined error`.

Design philosophy

FileMaker can return response headers from `Insert from URL` using the cURL `-D` option, but using the HTTP status in subsequent script logic normally requires extracting and interpreting the response line.

`VBZS_ResponseCode` reduces this to two immediately usable FileMaker variables:

```
$responseCode
$responseMessage
```

The function also handles response headers containing multiple HTTP responses by using the final response, which represents the result relevant to the completed request.

The response message is derived from a predefined mapping rather than depending on the text supplied by the remote server.

This keeps subsequent FileMaker script logic simple: the numeric response code can be used for decisions while the corresponding message is immediately available when a human-readable description is useful.

VBZS_TrimAll_CR

Description

`VBZS_TrimAll_CR` removes redundant carriage returns from a FileMaker value list.

Leading and trailing carriage returns are removed and multiple consecutive carriage returns between values are reduced to a single carriage return.

The result is a return-delimited list without empty values.

Syntax

```
VBZS_TrimAll_CR ( text )
```

Parameters

`text`

The text or return-delimited list from which redundant carriage returns should be removed.

Result

Returns the supplied text with redundant carriage returns removed.

For example:

```
VBZS_TrimAll_CR ( "¶¶A¶¶B¶C¶" )
```

returns:

```
A¶B¶C
```

Spaces

`VBZS_TrimAll_CR` only removes redundant carriage returns.

Extra spaces within or around values are not removed.

If spaces also need to be removed or normalized, use FileMaker's native `TrimAll()` function separately.

Example

Given:

```
¶
¶
A
¶
B
C
¶
```

the function removes the empty values and returns:

```
A
B
C
```

or, as FileMaker return-delimited text:

```
A¶B¶C
```

Notes

- Leading carriage returns are removed.
- Trailing carriage returns are removed.
- Consecutive carriage returns between values are reduced to one.
- Empty values are therefore removed from the list.
- Extra spaces are not removed.
- The order and contents of the remaining values are preserved.

Design philosophy

FileMaker value lists can easily acquire empty values when lists from different sources are combined or manipulated.

`VBZS_TrimAll_CR` provides a simple way to remove those empty values without modifying the contents of the remaining values.

The function deliberately operates only on carriage returns. Space handling is left to FileMaker's native `TrimAll()` function.

VBZS_UTC2Local

Description

`VBZS_UTC2Local` converts a UTC timestamp or numeric UTC time value to local time.

The conversion uses the supplied GMT timezone offset and automatically applies the built-in European Daylight Saving Time correction when applicable.

The function can also preserve fractional seconds, with the number of decimal places controlled by the `decimals` parameter.

Syntax

```
VBZS_UTC2Local ( UTC ; timezone ; decimals )
```

Parameters

`UTC`

The UTC time to convert.

The value can be supplied as a FileMaker timestamp or as a numeric UTC time value.

A common use is FileMaker's:

```
Get ( CurrentTimeUTCMicroseconds )
```

Dividing this value by `1000000` produces a UTC value expressed in seconds while preserving the fractional part:

```
Get ( CurrentTimeUTCMicroseconds ) / 1000000
```

This can be passed directly to `VBZS_UTC2Local`.

`timezone`

The normal GMT offset of the desired local timezone, expressed in hours.

The supported range is:

```
-12 ... +12
```

Fractional timezone offsets are supported.

For example:

```
1
```

represents GMT+1 during standard time.

Offsets containing fractions of an hour can also be used:

```
5.5
5.25
```

The function applies its built-in European Daylight Saving Time correction in addition to this normal GMT offset.

`decimals`

The number of decimal places to return for the fractional part of the seconds.

Supported values are:

```
0 ... 6
```

For example:

```
0
```

returns whole seconds, while:

```
6
```

returns microsecond precision when that precision is available in the supplied UTC value.

Result

Returns the UTC value converted to local time using:

1. the supplied normal GMT offset;
2. the built-in European Daylight Saving Time correction;
3. the requested number of fractional second digits.

The result uses the system's timestamp representation.

Example

```
VBZS_UTC2Local (
    Get ( CurrentTimeUTCMicroseconds ) / 1000000 ;
    1 ;
    6
)
```

may return:

```
06/10/2026 22:38:45,697535
```

In this example:

- the UTC value is obtained from `Get ( CurrentTimeUTCMicroseconds )`;
- dividing by `1000000` converts the microsecond value to seconds while retaining the fractional part;
- `timezone = 1` specifies a normal offset of GMT+1;
- the built-in Daylight Saving Time calculation determines whether an additional hour applies;
- `decimals = 6` retains six digits of fractional seconds.

Fractional seconds

The `decimals` parameter determines how many fractional second digits are included in the result.

For example, with a local time equivalent to:

```
22:38:45,697535
```

using:

```
decimals = 6
```

preserves:

```
697535
```

as the fractional part of the second.

Using:

```
decimals = 0
```

returns the timestamp without fractional seconds.

The supported range is 0 through 6 decimal places.

Fractional timezone offsets

Timezone offsets are not limited to whole hours.

For example:

```
timezone = 5.5
```

represents an offset of five hours and thirty minutes from GMT before any applicable built-in Daylight Saving Time correction.

Quarter-hour offsets can also be represented:

```
timezone = 5.25
```

This makes the timezone calculation suitable for locations whose normal GMT offset does not consist of complete hours.

Daylight Saving Time

`VBZS_UTC2Local` contains built-in Daylight Saving Time rules for Europe.

The supplied `timezone` represents the normal GMT offset. During Daylight Saving Time, the function adds one additional hour when the built-in DST rules determine that summer time applies.

For example:

```
timezone = 1
```

represents:

```
UTC+01:00
```

during standard time and:

```
UTC+02:00
```

during Daylight Saving Time.

The built-in transitions are:

- Daylight Saving Time starts on the last Sunday of March at `00:00 UTC`.
- Daylight Saving Time ends on the last Sunday of October at `01:00 UTC`.

> **Important:** The automatic Daylight Saving Time calculation uses European DST rules. Regions using different DST rules cannot rely on the automatic DST correction for local-time conversion.

Relationship with VBZS_Local2UTC

`VBZS_UTC2Local` performs the opposite conversion to `VBZS_Local2UTC`.

Conceptually:

```
VBZS_Local2UTC

local timestamp
      +
timezone
      +
European DST
      ↓
UTC
```

while:

```
VBZS_UTC2Local

UTC
      +
timezone
      +
European DST
      ↓
local timestamp
```

The functions can therefore be used for opposite sides of timestamp conversion when the same timezone and European DST rules apply.

Notes

- `UTC` can be supplied as a FileMaker timestamp or numeric UTC time value.
- `timezone` represents the normal GMT offset in hours.
- `timezone` can contain fractional hours.
- The documented timezone range is `-12` through `+12`.
- European Daylight Saving Time is calculated automatically.

- The built-in DST calculation is not suitable for regions using different DST transition rules.
- `decimals` specifies the number of fractional second digits.
- The documented range for `decimals` is `0` through `6`.
- `decimals = 0` returns whole seconds.
- `decimals = 6` can retain microsecond precision when present in the input.
- The result is returned using the system timestamp representation.

Design philosophy

FileMaker provides:

```
Get ( CurrentTimeUTCMicroseconds )
```

which makes a precise UTC time available, but applications often need that value represented as a local timestamp.

`VBZS_UTC2Local` combines the UTC value with a GMT timezone offset and the built-in European Daylight Saving Time rules to produce that local timestamp.

Support for fractional timezone offsets allows the same calculation mechanism to work with timezones that differ from GMT by whole, half or quarter hours.

The `decimals` parameter allows the caller to decide how much of the available sub-second precision should be retained, from normal whole-second timestamps through microsecond precision.

VBZS_Version

Description

`VBZS_Version` returns the version number of the installed VBZS plug-in.

The version can be returned in two different representations:

- a normal human-readable version;
- a fixed-length representation intended for version checking and auto-update routines.

Syntax

```
VBZS_Version ( parameter )
```

Parameters

`parameter`

Determines how the version number is returned.

Use an empty parameter for the normal version representation:

```
""
```

Use either:

```
1
```

or:

```
"autoupdate"
```

for the fixed-length auto-update representation.

Standard version representation

With an empty parameter:

```
VBZS_Version ( "" )
```

the version is returned in its normal human-readable form.

For example:

```
0.9.1.1
```

The VBZS version consists of four numeric components separated by periods.

Auto-update representation

Using:

```
VBZS_Version ( 1 )
```

or:

```
VBZS_Version ( "autoupdate" )
```

returns the same version as a fixed 8-digit representation.

For example:

```
0.9.1.1
```

is returned as:

```
00090101
```

Each of the four version components occupies two positions:

```
0  →  00
9  →  09
1  →  01
1  →  01
```

Together:

```
00 09 01 01
```

becomes:

```
00090101
```

Version comparison

The fixed-length representation is intended to make version checking easier.

For example, these versions:

```
0.9.1.1
0.9.3.4
0.10.0.0
1.0.0.0
```

are represented as:

```
00090101
00090304
00100000
01000000
```

Because every version component always occupies two positions, the representations can be compared without first extracting the individual components from the normal dotted version number.

This makes the format useful for determining whether an installed plug-in should be replaced by a newer version.

Example

To display or retrieve the normal version:

```
VBZS_Version ( "" )
```

returns, for example:

```
0.9.1.1
```

For version checking:

```
VBZS_Version ( 1 )
```

returns:

```
00090101
```

The same representation can be requested using:

```
VBZS_Version ( "autoupdate" )
```

which also returns:

```
00090101
```

Notes

- The version consists of four numeric components.
- Each component has a range of `0` through `99`.
- The normal representation separates the components with periods.
- An empty parameter returns the normal version representation.
- `1` returns the fixed-length auto-update representation.
- `"autoupdate"` also returns the fixed-length auto-update representation.
- The auto-update representation always contains eight digits.
- Each version component occupies exactly two positions.
- Leading zeros are added where required.
- The fixed-length representation is intended for version checking and update routines.

Design philosophy

A normal dotted version number is easy to read:

```
0.9.1.1
```

but is less convenient when a FileMaker solution needs to determine whether the installed plug-in should be replaced by another version.

`VBZS_Version` therefore provides a second representation specifically for version checking.

The four version components are converted into fixed-width two-digit values:

```
0.9.1.1
```

becomes:

```
00090101
```

This keeps the normal version readable for users while providing a predictable representation for automated version comparisons.

VBZS_WeekFiscalToDate

Description

`VBZS_WeekFiscalToDate` calculates the date of the first day of a specified week number in a given year.

The function uses ISO-style week-numbering rules, but allows the first day of the week to be configured.

This makes it possible to use the conventional European Monday-based week as well as week structures that start on another day.

Syntax

```
VBZS_WeekFiscalToDate ( Yearnr ; WeeknrOfYear ; Firstweekday )
```

Parameters

`Yearnr`

The year for which the week number should be calculated.

The supported range is:

```
1 ... 4000
```

`WeeknrOfYear`

The week number for which the starting date should be returned.

`WeeknrOfYear` can be any natural number. It is not explicitly limited to 52 or 53.

The practical limit is determined by the resulting FileMaker date.

`Firstweekday`

Defines which day is considered the first day of the week.

The numbering follows FileMaker's weekday convention:

Value	Day
1	Sunday
2	Monday
3	Tuesday
4	Wednesday
5	Thursday
6	Friday
7	Saturday

For the conventional European ISO week:

```
Firstweekday = 2
```

which makes Monday the first day of the week.

Result

Returns the FileMaker date representing the first day of the requested week.

The result depends on:

- the supplied year;
- the requested week number;
- the configured first day of the week.

Example

```
VBZS_WeekFiscalToDate (
    2026 ;
    41 ;
    2
)
```

returns:

```
05-10-2026
```

With `Firstweekday = 2`, Monday is the first day of the week.

October 5, 2026 is therefore the first day of week 41.

Week numbering

`VBZS_WeekFiscalToDate` uses the ISO principle for determining which week belongs to the new year.

Week 1 is the week containing at least four days of the new year.

If January 1 falls within the first four days of the configured week, that complete week is considered week 1 of the new year.

This means that the first days of week 1 can actually occur in December of the preceding calendar year.

If January 1 falls within the final three days of the configured week, January 1 is considered part of the last week of the preceding week-numbering year.

Monday-based ISO week

With:

```
Firstweekday = 2
```

the week starts on Monday.

The week containing January 1 belongs to the new year when January 1 occurs on:

```
Monday
Tuesday
Wednesday
Thursday
```

If January 1 occurs on:

```
Friday
Saturday
Sunday
```

that date belongs to the last week of the preceding week-numbering year.

This corresponds to the conventional ISO week-numbering system.

Configurable first weekday

The same 4/3 rule is applied when another first weekday is selected.

For example:

```
Firstweekday = 1
```

defines Sunday as the beginning of the week.

The week boundaries move accordingly, while the rule determining week 1 remains the same:

```
First four days of the week
    → week belongs to the new year

Last three days of the week
    → week belongs to the preceding year
```

This allows the function to calculate week-based dates for calendars where the week does not start on Monday.

Week-numbering year versus calendar year

The week-numbering year and calendar year do not always start on the same date.

For example, week 1 of a new year can begin during the final days of December.

Likewise, the first days of January may still belong to the final week of the preceding week-numbering year.

`Yearnr` therefore identifies the week-numbering year used in the calculation, while the resulting FileMaker date may fall in the preceding calendar year.

Notes

- `Yearnr` supports years from `1` through `4000`.
- `WeeknrOfYear` is not restricted to 52 or 53.
- `Firstweekday` must be between `1` and `7`.
- FileMaker weekday numbering is used: Sunday = `1` through Saturday = `7`.
- `Firstweekday = 2` produces the conventional Monday-based European ISO week.
- Week 1 contains at least four days of the new year.
- A week containing January 1 within its final three days belongs to the preceding week-numbering year.
- The function returns the first day of the requested week as a FileMaker date.

Design philosophy

Week numbers become less straightforward around the beginning and end of a calendar year.

In addition, not every application considers the same weekday to be the beginning of a week.

`VBZS_WeekFiscalToDate` separates these two concepts.

The ISO-style 4/3 rule determines which week belongs to a particular year, while `Firstweekday` determines where each week begins.

With Monday as the first weekday, the function follows the conventional European ISO week structure.

By changing `Firstweekday`, the same calculation can also be used for fiscal or organisational calendars whose weeks start on a different day.

VBZS_XChildren

Description

`VBZS_XChildren` returns the names of the direct child elements in a supplied XML element.

The function is intended to help FileMaker developers explore the structure of an XML document one level at a time.

XML namespace prefixes are returned as part of the element name. Attributes belonging to a child element are listed immediately below that element and are prefixed with `@`.

Syntax

```
VBZS_XChildren ( xml )
```

Parameters

`xml`

A well-formed XML document or XML element to inspect.

Result

Returns a return-delimited list containing:

- the names of all direct child elements;
- the namespace prefix of each element, when present;
- the attributes belonging to each child element.

Attributes are prefixed with `@` and appear directly below the element to which they belong.

The function only returns direct children. It does not recursively list descendants at deeper levels.

Example

Given the following XML:

```
<Invoice>
    <cbc:ID schemeID="invoice">12345</cbc:ID>
    <cac:Customer>
        <cbc:Name>Acme BV</cbc:Name>
    </cac:Customer>
</Invoice>
```

The following calculation:

```
VBZS_XChildren ( xml )
```

returns:

```
cbc:ID
@schemeID
cac:Customer
```

The `cbc:Name` element is not included because it is not a direct child of `Invoice`. It is a child of `cac:Customer`.

Exploring an unknown XML document

`VBZS_XChildren` is designed to help explore unfamiliar XML without first knowing its complete structure.

A typical workflow is:

1. Supply the complete XML document to `VBZS_XChildren`.
2. Review the elements at the current level.
3. Select the relevant element.
4. Inspect that element or construct an XPath to its children.
5. Repeat until the required value has been located.

For example, an initial call might reveal:

```
cac:AccountingCustomerParty
```

Inspecting that part of the XML may then reveal:

```
cac:Party
```

The next level may contain:

```
cbc:Name
cbc:CompanyID
```

Once the required element has been identified, `VBZS_XPath` can retrieve its value from the original XML document.

Namespace handling

VBZS treats an XML namespace prefix as part of the element name from the FileMaker developer's point of view.

For example:

```
<cac:Customer>
```

is returned as:

```
cac:Customer
```

There is no separate namespace registration step.

This allows developers to work with the element names exactly as they appear in the XML document.

Important

The supplied XML must be well-formed.

`VBZS_XChildren` examines only the direct children of the supplied XML element. To inspect a deeper level, supply the relevant well-formed element or use `VBZS_XPath` to address the required path in the original XML document.

Related functions

- `VBZS_XPath`

VBZS_XPath

Description

`VBZS_XPath` evaluates an XPath expression against a supplied XML document and returns the raw content selected by that expression.

The function is designed to make XML navigation straightforward for FileMaker developers. Namespace prefixes can be used directly in the XPath as they appear in the XML document. No separate namespace registration is required.

Syntax

```
VBZS_XPath ( xml ; xpath )
```

Parameters

`xml`

The well-formed XML document to query.

`xpath`

The XPath expression identifying the element or content to return.

Result

Returns the raw content selected by the XPath expression.

If the selected element contains plain text or a number, that value is returned directly.

If the selected element contains child elements, their raw XML content is returned. This output is not guaranteed to be a well-formed XML document.

Basic example

Given:

```
<Invoice>
    <cbc:ID>12345</cbc:ID>
</Invoice>
```

The following calculation:

```
VBZS_XPath (
    xml ;
    "/Invoice/cbc:ID"
)
```

returns:

```
12345
```

Namespace handling

Namespace prefixes can be used directly in the XPath expression.

Given an XML structure such as:

```
<Invoice>
    <cac:AccountingCustomerParty>
        <cac:Party>
            <cbc:Name>Acme BV</cbc:Name>
        </cac:Party>
    </cac:AccountingCustomerParty>
</Invoice>
```

the value can be retrieved using:

```
VBZS_XPath (
    xml ;
    "/Invoice/cac:AccountingCustomerParty/cac:Party/cbc:Name"
)
```

The result is:

```
Acme BV
```

There is no need to register `cac` or `cbc` separately. Use the prefixes exactly as they appear in the XML document.

Important: the input must be well-formed XML

`VBZS_XPath` always requires well-formed XML as its input.

This requirement applies to every call, including calls that use the result of another function.

A fragment containing multiple top-level elements is not well-formed XML and cannot be passed directly to `VBZS_XPath`.

Important: the result contains raw content

`VBZS_XPath` returns the raw content of the selected element. It does not automatically wrap that content in a new root element.

For example, given:

```
<cac:Party>
    <cbc:Name>Acme BV</cbc:Name>
    <cbc:CompanyID>12345</cbc:CompanyID>
</cac:Party>
```

selecting the content of `cac:Party` may return:

```
<cbc:Name>Acme BV</cbc:Name>
<cbc:CompanyID>12345</cbc:CompanyID>
```

This result contains two top-level elements. It is therefore an XML fragment, not a well-formed XML document.

The result cannot be used directly as the XML parameter of another `VBZS_XPath` call.

> **Important:** `VBZS_XPath` requires well-formed XML as input, but its output is deliberately the raw content of the selected element. When a selected element contains child elements, the returned content is not guaranteed to be well-formed XML.

Working with complex elements

Do not assume that every result returned by `VBZS_XPath` can be used as XML input for another call.

When an element contains child elements:

1. Use `VBZS_XChildren` to discover the available elements.
2. Construct the complete XPath to the required value.
3. Apply that XPath to the original well-formed XML document.

For example:

```
VBZS_XPath (
    xml ;
    "/Invoice/cac:AccountingCustomerParty/cac:Party/cbc:CompanyID"
)
```

returns:

```
12345
```

This is preferable to extracting `cac:Party` first and then trying to query the returned XML fragment.

Recommended workflow

Use the two XML functions together:

1. Use `VBZS_XChildren` to discover the structure of an unfamiliar XML document.
2. Continue exploring the relevant child elements.
3. Construct the complete path to the required value.
4. Use `VBZS_XPath` with the original well-formed XML document.
5. Use the returned text or number directly in FileMaker.

In short:

```
Unknown XML
    ↓
VBZS_XChildren
    ↓
Discover the structure
    ↓
Construct the required XPath
    ↓
VBZS_XPath
    ↓
Retrieve the required value
```

Design philosophy

`VBZS_XPath` is primarily intended to retrieve data from XML. It is not intended to extract a section of an XML document and automatically turn that section into a new XML document.

The function therefore returns the content that was found without adding a root element or otherwise rewriting the XML.

This keeps the result predictable and avoids silently modifying the original XML content.

Related functions

- `VBZS_XChildren`

VBZS_YearFiscal

Description

`VBZS_YearFiscal` returns the week-numbering year to which a given date belongs.

For most dates, the returned year is identical to the calendar year. The distinction becomes relevant during the first and last week of a year, where a week can span two different calendar years.

The function uses ISO-style week-numbering rules while allowing the first day of the week to be configured.

Syntax

```
VBZS_YearFiscal ( date ; startingDay )
```

Parameters

`date`

The FileMaker date for which the week-numbering year should be determined.

`startingDay`

Defines which day is considered the first day of the week.

The numbering follows FileMaker's weekday convention:

Value	Day
1	Sunday
2	Monday
3	Tuesday
4	Wednesday
5	Thursday
6	Friday
7	Saturday

For the conventional European ISO week:

```
startingDay = 2
```

which makes Monday the first day of the week.

Result

Returns the year to which the week containing `date` belongs.

The result may differ from the calendar year of `date` during the first or last days of a calendar year.

Example

```
VBZS_YearFiscal (
    Date ( 1 ; 1 ; 2027 ) ;
    2
)
```

returns:

```
2026
```

January 1, 2027 is a Friday.

With Monday as the first day of the week, Friday falls within the final three days of that week. The complete week therefore belongs to the preceding week-numbering year:

```
2026
```

Week-numbering rule

`VBZS_YearFiscal` uses the ISO principle that week 1 must contain at least four days of the new year.

If January 1 occurs within the first four days of the configured week, that complete week belongs to the new year.

If January 1 occurs within the final three days of the configured week, that week belongs to the preceding week-numbering year.

This means that calendar year and week-numbering year are not always identical around the beginning and end of a year.

Monday-based ISO year

With:

```
startingDay = 2
```

the week begins on Monday.

If January 1 occurs on:

```
Monday
Tuesday
Wednesday
Thursday
```

the week belongs to the new week-numbering year.

If January 1 occurs on:

```
Friday
Saturday
Sunday
```

the date belongs to the final week of the preceding week-numbering year.

This corresponds to the conventional ISO week-numbering system.

Dates at the end of December

The reverse situation can occur at the end of December.

A date in the final days of December may already belong to week 1 of the following week-numbering year.

The function therefore does not simply return:

```
Year ( date )
```

Instead, it determines which week contains the supplied date and then returns the year to which that complete week belongs.

Configurable first weekday

The same 4/3 rule is used when a different first weekday is selected.

For example:

```
startingDay = 1
```

makes Sunday the first day of the week.

The week boundaries move accordingly, but the principle remains the same:

```
At least four days in the new year
    → week belongs to the new year

Three or fewer days in the new year
    → week belongs to the preceding year
```

This makes the function useful for week-based or fiscal calendars that do not use Monday as the first day of the week.

Relationship with VBZS_WeekFiscalToDate

`VBZS_YearFiscal` uses the same week-numbering principle as `VBZS_WeekFiscalToDate`.

The two functions approach the calculation from opposite directions.

`VBZS_YearFiscal` determines:

```
date
    ↓
week-numbering year
```

while `VBZS_WeekFiscalToDate` determines:

```
week-numbering year
+
week number
+
first weekday
    ↓
date
```

Using the same `startingDay` / `Firstweekday` convention keeps both calculations consistent.

Notes

- The function returns the week-numbering year for a FileMaker date.
- The result normally equals the calendar year.
- Differences occur around the first and last week of a calendar year.
- `startingDay` defines the first day of the week.
- FileMaker weekday numbering is used: Sunday = `1` through Saturday = `7`.
- `startingDay = 2` produces the conventional Monday-based ISO week structure.
- A week belongs to the new year when it contains at least four days of that year.
- The returned year may therefore differ from `Year ( date )`.

Design philosophy

For most dates, determining the year is as simple as using FileMaker's:

```
Year ( date )
```

Week-based calendars introduce an exception around the transition between calendar years.

The first days of January can still belong to the final week of the preceding year, while the final days of December can already belong to week 1 of the following year.

`VBZS_YearFiscal` handles this boundary and returns the year associated with the complete week rather than simply the calendar year of the supplied date.

Together with `VBZS_WeekFiscalToDate`, this provides consistent handling of week-based dates using either the normal Monday-based ISO calendar or another configured first day of the week.